

Three Orthogonal Failure Axes in Hybrid AR / Diffusion Reasoning, with Trainable Fixes for Two of Them

Eren Akbulut

Draft, April 2026

Abstract

Hybrid pipelines that combine an autoregressive (AR) language model with a discrete-diffusion language model (DDLm) for chain-of-thought reasoning have produced mixed results in the literature, with conflicting reports on whether AR planning helps DDLm denoising and whether DDLm sampling diversity is useful. This paper is about the *substrate properties* of current-generation DDLms that explain those conflicts — not about whether the hybrid stack beats a parameter-matched monolithic AR baseline on a math benchmark, which on GSM8K [6] at the AR scales we use as planners it does not. We argue the conflicts in the literature collapse once the failure surface is decomposed: hybrid AR/DDLM reasoning fails along *at least three orthogonal axes* — interface-format brittleness, planner-content trust (which we characterize as a *phenomenon* over a two-rung capacity sweep, not yet a fully-resolved axis), and sampling-diversity preservation — that can be characterized and intervened on independently. On GSM8K [6] with LLADA-8B-Instruct [15] and QWEN2.5-{0.5B,1.5B}-Instruct [17] we show: (i) interface-format brittleness is fixable with a small ($r=8$) [9] prefix-robustness LoRA, with a measurable capacity tradeoff between no-prefix accuracy and branch-vote ceiling; (ii) planner-content trust is capacity-dependent in opposite directions across two planners — a 0.5B planner improves at higher capacity (+5 pp), while a 1.5B planner regresses by 13 pp — with the caveat that two capacity rungs establish the inversion is real but a third rung is needed to fully resolve the curve; (iii) format-augmented LoRA training *expands* sampling diversity (5/5-branch agreement 51.5% $\rightarrow$ 47.5%), rather than collapsing it; (iv) consensus distillation into the diffusion model is design-sensitive, not architecture-limited: late-block answer-span surgery fails to bridge the `c2c`-vs-`cmaj` gap (`c2c`= 70.5% across two iterations); earlier-block full-response surgery recovers (`c2c`= 79.0%, within sampling error of the 80% pre-registered target). Disentangling ablations attribute most of the lift (+4 pp) to where commit fires in the diffusion schedule, with full-response training a smaller secondary effect (+2 pp). Branch aggregation (`cmajc`) on top of the `c2c`-fixed v3 design reaches 82.5%, +3.5 pp over base test `cmaj`, indicating param-time and compute-time consensus mechanisms compose rather than substitute. A QWEN2.5-0.5B self-consistency baseline [18] ($b=5, t=0.7$) reaches only 40.5% on the same eval, vs LLADA’s 79–82%, ruling out generic self-consistency as the explanation for the diffusion `cmaj` advantage. Finally, `cmaj` itself leaves 8–12 pp on the table relative to an oracle ceiling, and this gap is structural: 17 peer-class verifiers (text-bag, 0.5B–72B chat-LMs, embedding and reward models, process-feature MLPs, step-PRMs, symbolic arithmetic) all underperform majority vote, while a frontier-judge baseline (Claude Sonnet 4.5 with chain-of-thought) closes 86% of the gap (`cmaj` 79.1% $\rightarrow$ judge 85.3%, oracle 86.3% on $N=500$). The mechanism is that LLADA’s GSM8K failure mode is problem comprehension, not arithmetic.

A K2 sub-block ablation on the v3 commit adapter produces an inverted-U around the v3 default (COMMIT_N_BLOCKS: $k=3$ peak, $k=0$ adapter-off -1.7 pp, $k=4$ always-on -3.2 pp), locating commit-LoRA as a single discrete inference-time schedule toggle at the load-bearing

sub-block-1 boundary — a strictly weaker mechanism than continuous schedule-aware adapters (TC-LoRA [2], TimeStep Master [3]). Across the parallel sub-problems of test-time aggregation, a consistent pathology emerges: an offline-replay per-problem mode router with surface text features (−10 pp vs best fixed), a schedule-weighted temporal-SC voting variant (−4 pp vs `cmajc`-vote, against the positive-direction prior of Zhang et al. [21]), a KL-anchored schedule-RLHF mid-pilot on commit-LoRA (within CI on MATH-500 $N=50$ but with a directional reversal of the unanchored mini-pilot), and a LLaDA-1.5 base swap with frozen v3 LoRAs (−2 pp) all land as honest negatives or within-CI signals at our substrate scale. We surface these alongside a parameter-matched plain Qwen-2.5-7B AR baseline (which reaches $\sim 86.5\%$, beating the hybrid `cmajc` at fewer active parameters) as disclosure rather than as targets. All datasets and adapters are public on the Hugging Face Hub. Total compute spend across all experiments is approximately \$20.5 on a single RTX 4090 substrate (Appendix B).

1 Introduction

Hybrid pipelines that pair an autoregressive (AR) language model with a discrete-diffusion language model (DDLm) for chain-of-thought reasoning have produced contradictory empirical reports. Some authors find that an AR planner improves DDLm denoising [20, 22]; others find that the same pattern degrades it [10]; and reports on whether DDLm sampling diversity is preserved or collapses under fine-tuning likewise disagree [12]. The picture is muddled further by the fact that recent DDLms [5, 8, 15] share a tokenizer family and a semi-AR sampler but are evaluated on different benchmarks and at different prefix conditions, so the underlying disagreement is hard to isolate.

We started this project from a narrow observation in our own prior diagnostic work (E4): on LLADA-8B-Instruct [15], prepending a literal "Plan: " string to a GSM8K question [6] costs 8 pp of accuracy versus the no-prefix baseline (74% $\rightarrow$ 66%), even though the prefix contains no content at all. The damage is format-shape, not plan quality. This single number was inconsistent with the simplest "AR plan helps DDLm" story and is also inconsistent with the simplest "AR plan hurts DDLm" story: the cost is real, but the mechanism is not the planner. It is the prefix surface itself.

This paper argues that the conflicts in the literature collapse once the failure surface is decomposed into orthogonal axes that can be measured and intervened on independently. We identify three.

1. **Interface-format brittleness:** how much the DDLm’s accuracy drops when the question is wrapped in any plan-shaped scaffold, content-free or otherwise.
2. **Planner-content trust:** how much the DDLm uses the actual content of an upstream AR plan, conditional on the prefix shape being absorbed.
3. **Sampling-diversity preservation:** whether fine-tuning the DDLm collapses, preserves, or expands the variety of branches produced by stochastic sampling, on which compute-time consensus mechanisms such as self-consistency [18] depend.

On GSM8K-test ($N=200$) with LLADA-8B-Instruct and QWEN2.5- $\{0.5B, 1.5B\}$ -Instruct [17] planners, we characterize each axis with a small ($r=8$) LoRA [9, 14] and report the following.

First, interface-format brittleness is trainably fixable. A prefix-robust LoRA on LLADA with full FFN module coverage (v3, 22M trainable parameters) lifts no-prefix C2 from 74% $\rightarrow$ 73.0% (within sampling error) while pulling the empty-prefix "Plan: " condition from 66% $\rightarrow$ 74%, flattening the no-prefix-vs-empty-prefix gap from 8 pp to within 1 pp. A partial-coverage variant

(v2, 4/7 modules, 10M parameters) achieves slightly weaker single-shot but stronger branch-vote performance, exposing a no-prefix-vs-**cmaj** capacity tradeoff.

Second, planner-content trust is capacity-dependent in opposite directions across planners. At v2 capacity, a 1.5B AR planner helps (60% $\rightarrow$ 67%, +7 pp) and a 0.5B planner still hurts (64% $\rightarrow$ 60%). At v3 capacity this inverts: the 0.5B planner improves (64% $\rightarrow$ 65%, +5 pp) and the 1.5B planner catastrophically regresses (60% $\rightarrow$ 54%, -13 pp). Capacity differentially modulates how much the model trusts upstream content, and the optimal capacity depends on the planner.

Third, format-augmented training expands sampling diversity rather than collapsing it. After the prefix-robust LoRA (v2) plus a late-block commit adapter, the rate at which all 5 stochastic branches agree drops from 51.5% to 47.5% on test, and mean unique answers per problem rises from 1.83 to 2.07 (+13%). **cmaj** accuracy holds at 81.5% on dev, and **cmajc** reaches 82.0% (commit v2) / 82.5% (commit v3) on test. Diversity goes up, accuracy is preserved or improved.

Fourth, weight-space distillation of the consensus mechanism (**cmaj** into a single forward pass) is design-sensitive in a way that is easy to miss. A late-block, answer-span commit adapter fails to bridge the 6 pp gap between greedy **C2** and **cmaj**, plateauing at **c2c**= 70.5% across two iterations and a 3.25 $\times$ capacity bump. A multi-block, full-response variant recovers **c2c**= 79.0%, within sampling error of the 80% pre-registered target. Disentangling ablations attribute +4 pp of the lift to where commit fires in the diffusion schedule and +2 pp to full-response supervision; full-response training on its own (with last-block-only commit) adds nothing.

Contributions.

1. **A three-axis decomposition of hybrid AR/DDLM reasoning failure.** Interface-format brittleness, planner-content trust, and sampling-diversity preservation are independently measurable on GSM8K with LLADA and QWEN2.5- $\{0.5\text{B}, 1.5\text{B}\}$. The decomposition reconciles conflicting reports in the literature. Axis 2 is characterized as a *phenomenon* over a two-rung capacity sweep, not yet a fully-resolved axis on equal footing with axes 1 and 3 (§3–§5, §11).
2. **Trainable interventions on two of the three axes.** Interface-format brittleness is fixable with a small ($r=8$) prefix-robustness LoRA, with a measurable capacity tradeoff: v2 (4/7 FFN modules, 10M parameters) and v3 (7/7 FFN modules, 22M parameters) trade +2.5 pp on no-prefix C2 for -2.0 pp on **cmaj** branch-vote, locating a single-shot-vs-ensemble Pareto frontier (§3). Format-augmented training also *expands* sampling diversity rather than collapsing it (5/5-branch agreement 51.5% $\rightarrow$ 47.5%, mean unique answers 1.83 $\rightarrow$ 2.07), inverting the canonical fine-tuning prediction (§5). A separate commit-LoRA distills the consensus mechanism into a single forward pass (§6).
3. **Consensus distillation into a DDLM is design-sensitive, not architecture-limited, and the mechanism is block coverage.** Late-block answer-span surgery plateaus at $\text{c2c} = 70.5\%$ across two iterations and a $3.25\times$ capacity bump — a clean negative whose surface reading is “the adapter cannot do this”. A multi-block ($n=3$) full-response variant recovers $\text{c2c} = 79.0\%$, within sampling error of an 80% pre-registered target. Disentangling ablations attribute most of the lift (+4 pp) to where commit fires in the diffusion schedule and +2 pp to full-response training; full-response training alone (with last-block-only commit) adds +0.0 pp. Two design errors mask each other; block coverage is the dominant lever (§6).
4. **cmaj aggregation is structurally lossy by 8–12 pp; the failure mode is problem comprehension, not arithmetic.** The oracle ceiling exceeds majority-vote accuracy by 8–12 pp across four LoRA configurations and three seeds. The gap survives every peer-class verifier we try: 17 architectures (text-bag, 0.5B–72B chat-LMs, embedding and reward models, process-feature MLPs, step-PRMs, symbolic arithmetic) all underperform majority vote. A frontier judge with chain-of-thought (Claude Sonnet 4.5 [4]) closes 86% of the gap (**cmaj** 79.1% $\rightarrow$ judge 85.3%, oracle 86.3% on $N=500$). Hand inspection of the 15 **cmaj**-failed/oracle-recoverable problems shows wrong branches contain locally correct arithmetic applied to incorrect problem setups, predicting the entire verifier leaderboard from a single diagnosis (§8).

Beyond these four headline contributions, additional findings and honest negatives are surfaced as prose inside the relevant sections rather than as standalone bullets: a K2 schedule-toggle ablation locating commit-LoRA as a discrete inference-time toggle on block-structured mask diffusion (§9), an offline-replay per-problem mode router that hits a pre-registered LOSS threshold (§10), three parallel aggregation negatives (temporal-SC voting, KL-anchored schedule-RLHF, LLADA-1.5 base swap; §11.2), a parameter-matched plain-Qwen-7B AR baseline that beats the hybrid **cmajc** at fewer active parameters (§11.3), a QWEN2.5-0.5B self-consistency rebuttal that rules out generic self-consistency as the explanation for the diffusion **cmaj** advantage (§7), and the compositionality of param-time and compute-time consensus (**cmajc** = 82.5% vs base test **cmaj** = 79.0%, +3.5 pp; §6).

Framing: substrate properties, not benchmark wins. This paper is about *substrate properties* of current-generation discrete-diffusion language models for chain-of-thought reasoning — what fails, what is fixable, what is structural — not about whether a hybrid AR/DDLM stack beats a parameter-matched monolithic AR baseline on a math benchmark. At the AR scales we use as planners, on this benchmark, it does not: a plain Qwen-2.5-7B AR baseline reaches $\sim 86.5\%$ on GSM8K-test, beating our hybrid `cmajc` of 82.5% at fewer active parameters, and hybrid accuracy is approximately invariant (82–83%) across a $14\times$ planner-scale range from Qwen-0.5B to Qwen-7B (§11.3). The natural read is that the substrate-level findings reported here should generalize to AR/DDLM stacks where the diffusion model is closer to AR peer-class scale, and that the practical accuracy claim of *this* hybrid against parameter-matched AR is benchmark- and scale-bounded. We take that bet upfront so it does not ambush the discussion. Two of the three axes admit trainable interventions (§3, §6); the third is characterized as a phenomenon over a two-rung capacity sweep (§4) and we are explicit that a third rung would upgrade it from a phenomenon to a fully-resolved axis.

Scope and statistical caveats. All numbers are measured on GSM8K-test at $N=200$, single-seed 0 unless otherwise noted, with LLADA-8B-Instruct at $k=64$ denoising steps and a fixed regex answer extractor that tolerates both `Answer:` and `####` formats. At $N=200$, binomial sampling noise is approximately $\pm 5\text{--}7$ pp, which is the right scale to keep in mind when reading the deltas reported in §3–§6. The multi-seed Phase-1 `cmajc` headline is locked at $82.2\% \pm 0.29$ pp (σ across 3 seeds at $N=100$), tighter than the per-seed Wilson interval at $N=100$; the other single-seed deltas in the body should be read as point estimates and are flagged as such throughout. The voting-rule gap section (§8) additionally uses an $N=500$ extension (200 frozen indices + 300 fresh) for the frontier-judge evaluation; the substrate change moves the base `cmaj` only from 79.0% to 79.1%, well within sampling noise. All experiments use a single benchmark (GSM8K-test) and a single base diffusion model (LLADA-8B-Instruct); a multi-task replication (e.g. MATH-Easy, ARC-Challenge) and a multi-DDLM substrate replication (BD3-LMs [5], DiffuLLaMA [8]) would harden every claim made here, and we say so plainly in §11-Limitations and Appendix E.

Reproducibility and cost. Total compute spend across all four phases of this work is approximately \$20.5 on a single RTX 4090 substrate at \$0.20/hr (Runpod), batched into ~ 100 pod-hours (Appendix B). All four LoRA adapters (Track 1 v2, Track 1 v3, Track 2 v2, Track 2 v3) and the three derived datasets are public on the Hugging Face Hub; the full pipeline re-runs end-to-end in under a single GPU-day. Lab-independent reproducibility on consumer hardware is itself part of the contribution: the substrate-level findings in this paper do not require a research-cluster budget to verify, and we view the cost ceiling as a feature of the design rather than a footnote. Appendix C lists exact training and verifier configurations; Appendix A reproduces the pre-registration scorecard, including a prediction violated in the unexpected (good) direction and a post-hoc verifier-aggregation prediction that hardens §8; Appendix E flags the remaining open exposures that we did not close before this draft.

2 Setup

Models. The diffusion model is LLADA-8B-Instruct [15], loaded in bf16 and used with $k=64$ denoising steps and a generation length of 128 tokens organized as four sub-blocks of 32 tokens each. Two AR planners are used without fine-tuning: QWEN2.5-0.5B-Instruct and QWEN2.5-1.5B-Instruct [17]. LLADA-8B and QWEN2.5-0.5B co-reside on a single RTX 4090 (24 GB) in bf16;

LLADA-8B and QWEN2.5-1.5B also co-reside. All model loading uses `trust_remote_code=True` for LLADA, and we pin `transformers==4.46.3` per the upstream recommendation.¹

Evaluation. GSM8K-test [6], $N=200$ frozen problem indices (stored as `e4/data/gsm8k_dev_200.json`), seed 0 unless otherwise noted. Numeric-answer extraction uses a regex on the last numeric span in the model output and tolerates both `Answer: <num>` and `#### <num>` formats. We report exact-match accuracy with no partial credit. The voting-rule gap section (§8) additionally uses an extended $N=500$ substrate (the original 200 frozen indices plus 300 fresh GSM8K-test indices, stored as `e4/data/gsm8k_dev_500.json`) for the frontier-judge evaluation; base `cmaj` on this larger substrate is 79.1%, within sampling error of the 79.0% at $N=200$. All other body numbers remain on $N=200$ frozen.

Statistical reporting. All headline percentages are computed over $N=200$ GSM8K-test problems unless otherwise noted. Reported confidence intervals are exact (Clopper–Pearson) binomial 95% intervals; we list them under each headline body table and in the pre-registration scorecard (Appendix A). Deltas between conditions are not CI’d individually, since paired-sample CIs require per-problem agreement data; per-problem eval JSONLs are released alongside the adapters for readers who want to compute paired intervals themselves.

Conditions. The eight conditions used throughout the paper are:

- **C2:** LLADA single-shot, no prefix.
- **C2hint:** LLADA with prefix `"Let’s think step by step.\n"`.
- **C2empty:** LLADA with literal prefix `"Plan: "` (no content).
- **C3p X:** AR planner X emits a plan, LLADA conditions on it and emits the answer; no AR finalize step. $X \in \{\text{QWEN2.5-0.5B}, \text{QWEN2.5-1.5B}\}$.
- **cmaj** ($b=5, t=0.7$): five stochastic LLADA branches, majority vote on extracted answers.
- **c2c:** C2 with commit-LoRA enabled (last sub-block only for v2; sub-blocks 2–4 of 4 for v3).
- **cmajc:** cmaj with commit-LoRA enabled per branch, then majority vote.

Adapters trained. We train four LoRA adapters on top of frozen LLADA-8B-Instruct.

Track 1 (prefix-robustness) trains a LoRA on the `eren23/sfumato-prefix-robust-gsm8k` dataset — 7,473 GSM8K-train problems crossed with eight prefix tiers (none / minimal / hint / xml / weak QWEN2.5-0.5B plan / medium QWEN2.5-1.5B plan / strong QWEN2.5-7B plan / oracle gold rationale), 59,784 rows total, stratified 95/5 train/validation. Two adapter variants:

- **v2** (10M trainable parameters): LoRA target modules `[q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]` — the standard Llama target list, which only matches 4 of LLADA’s 7 intended modules because LLADA’s `LLaDALlamaBlock` uses olmo-style names. $r=8, \alpha=16$, learning rate $5e-5$, 5,000 steps, $p_{\text{mask}} \sim U(0.1, 0.5)$.

¹Newer transformers releases break the LLADA `LLaDALlamaBlock` loader.

- **v3** (22M trainable parameters): LoRA target modules [q_proj, k_proj, v_proj, attn_out, ff_proj, up_proj, ff_out] — the LLADA-correct list, 7/7 match. All other hyperparameters identical to v2.

Track 2 (commit adapter) trains a smaller FFN-only LoRA that distils cmaj consensus into a single forward pass. The training data is `eren23/sfumato-commit-mixture-gsm8k`, 109 rows derived from running cmaj on 500 GSM8K-train problems and keeping three buckets: 46 rescue rows (consensus right, greedy wrong), 58 preserve-disagreement rows (both right but branches diverge), and 5 pure-agreement rows. Two variants:

- **v2** (13.6M trainable parameters): FFN-only target modules [ff_proj, up_proj, ff_out], layers 24–31 of 32, $r=8$, $\alpha=16$, learning rate $5e-5$, 10 epochs, $p_{\text{mask}} \sim U(0.3, 0.9)$, answer-span loss (CE on [answer_start, answer_end] only). Inference: commit on the last sub-block only (COMMIT_N_BLOCKS=1).
- **v3** (13.6M trainable parameters): identical to v2 except FULL_RESPONSE_LOSS=1 (CE on the full response span [prompt_len, response_end]) and inference uses COMMIT_N_BLOCKS=3 (commit on sub-blocks 2–4 of 4).

All four adapters and the two derived datasets above are public on the Hugging Face Hub under the `eren23/` namespace. Training is described per-script in Appendix C.

Pre-registration. Four predictions were registered in `e2/PROTOCOL.md` before training: (#1) `c2c` $\geq 80\%$, (#2) base cmaj within ± 1 pp of 80%, (#3) `cmajc` $\leq$ cmaj +1 pp (“no double-dip”), and (#4) planner-quality threshold shifts down with the LoRA. Appendix A reproduces the scorecard.

Hardware and compute. All training and evaluation runs on a single RTX 4090 spot or on-demand pod (Runpod, \$0.20/hr). Total spend across all four phases is approximately \$20.5; per-phase breakdown is in Appendix B.

3 Axis 1: Interface-Format Brittleness Is Trainably Fixable

The first axis is the simplest and the easiest to confuse for a planner-quality effect. Wrapping a GSM8K question in any plan-shaped scaffold — even an empty one — moves LLADA’s zero-shot accuracy. This section measures the base hierarchy, trains a small prefix-robust LoRA against it, and reports a capacity tradeoff that exposes the axis as separable from sampling diversity.

3.1 The Base Hierarchy

Table 1 reports base LLADA-8B-Instruct accuracy on GSM8K-test ($N=200$, $k=64$, seed 0) under five prefix conditions.

The empty-prefix “Plan: ” condition costs 8 pp versus no-prefix. LLADA-8B-Instruct was not trained to ignore plan-shaped scaffolds, and the literal token sequence — not the content — is enough to perturb its reasoning. This is the single observation that motivated the present paper.

Table 1: Base LLADA prefix-damage hierarchy on GSM8K-test, no LoRA. An empty "Plan: " prefix already costs 8pp; content-rich plans cost more, with the smaller QWEN2.5-1.5B planner producing the worst single-shot result. The damage tracks prefix shape, not plan quality.

Condition	Accuracy
C2 (no prefix)	74%
C2hint ("Let's think step by step.")	68%
C2empty ("Plan: ", no content)	66%
C3p Q-0.5B (weak plan)	64%
C3p Q-1.5B (medium plan)	60%

95% CIs (Clopper–Pearson, $N=200$): C2 [67.3, 79.9]; C2hint [61.1, 74.4]; C2empty [59.0, 72.5]; C3p Q-0.5B [56.9, 70.6]; C3p Q-1.5B [52.9, 66.8].

3.2 Track 1 v2: A Small LoRA Flattens Static-Prefix Damage

We train a small LoRA on the prefix-robust dataset described in §2. v2 uses target modules [q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj], the standard Llama list. Only 4 of these module names match LLADA’s actual block (q_proj, k_proj, v_proj, up_proj); the other three names are absent from LLADA’s LLaDALlamaBlock. The LoRA therefore attaches to 4 of 7 intended modules and trains $\approx 10\text{M}$ parameters. Hyperparameters: $r=8$, $\alpha=16$, learning rate $5\text{e}-5$, 5,000 steps, $p_{\text{mask}} \sim U(0.1, 0.5)$.

Table 2 reports v2 against base on the full prefix suite plus cmaj b=5 ($t=0.7$).

Table 2: Track 1 v2 versus base on GSM8K-test, $N=200$. Static prefix damage flattens (spread across {C2, C2hint, C2empty} drops from 8 pp to 3 pp). Q-1.5B plan helps; Q-0.5B plan still hurts. cmaj accuracy is preserved on dev.

Condition	Base	v2	Δ
C2	74%	70.5%	−3.5
C2hint	68%	73.5%	+5.5
C2empty	66%	73.0%	+7.0
C3p Q-0.5B	64%	60.0%	−4.0
C3p Q-1.5B	60%	67.0%	+7.0
cmaj b=5 (dev)	80%	81.5%	+1.5

95% CIs for v2 (Clopper–Pearson, $N=200$): C2 [63.7, 76.7]; C2hint [66.8, 79.5]; C2empty [66.3, 79.0]; C3p Q-0.5B [52.9, 66.8]; C3p Q-1.5B [60.0, 73.5]; cmaj b=5 (dev) [75.4, 86.6].

The static-prefix spread {C2, C2hint, C2empty} collapses from 8 pp (74 $\rightarrow$ 66) to 3 pp (70.5 $\rightarrow$ 73.5). The trained cmaj branch-vote ceiling is preserved at 81.5% on dev200. No-prefix C2 loses 3.5 pp; we discuss this tradeoff in the next subsection. The Q-0.5B planner condition still regresses (64 $\rightarrow$ 60), and the Q-1.5B planner improves (60 $\rightarrow$ 67): this directional split is the seed for the planner-content-trust axis (§4).

3.3 Track 1 v3: A Capacity Tradeoff

While debugging Track 2 (the commit adapter, §6), we discovered that LLADA’s LLaDALlamaBlock uses olmo-style FFN names (ff_proj, up_proj, ff_out) instead of the standard Llama (gate_proj,

up_proj, down_proj). v2’s target list matched only 4 of 7 intended modules. v3 retrains on the same data and hyperparameters with the LLADA-correct list [q_proj, k_proj, v_proj, attn_out, ff_proj, up_proj, ff_out] — a 7/7 match, with $\approx 22\text{M}$ trainable parameters ($2.2\times$ v2).

Table 3 reports v2 against v3 on the same eval.

Table 3: Track 1 v2 (4/7 modules, 10M params) against v3 (7/7 modules, 22M params) on GSM8K-test, $N=200$. Full-FFN coverage gains +2.5 pp on no-prefix C2 but loses -2.0 pp on cmaj branch-vote. The Q-0.5B/Q-1.5B planner split *inverts* between the two capacities. cmaj on test for v3 is reported apples-to-apples against base test cmaj of 79.0%.

Condition	Base	v2 (4/7)	v3 (7/7)	v2→v3
C2	74%	70.5%	73.0%	+2.5
C2hint	68%	73.5%	73.5%	0.0
C2empty	66%	73.0%	74.0%	+1.0
C3p Q-0.5B	64%	60.0%	65.0%	+5.0
C3p Q-1.5B	60%	67.0%	54.0%	-13.0
cmaj b=5	79% test, 80% dev	81.5% (dev)	79.5% (test)	-2.0

95% CIs for v3 (Clopper–Pearson, $N=200$): C2 [66.3, 79.0]; C2hint [66.8, 79.5]; C2empty [67.3, 79.9]; C3p Q-0.5B [58.0, 71.6]; C3p Q-1.5B [46.8, 61.1]; cmaj b=5 (test) [73.2, 84.9]. v2 column CIs are listed under Table 2; base column CIs under Table 1.

Two findings emerge from the v2-vs-v3 comparison.

(a) **A no-prefix-vs-cmaj capacity tradeoff.** Going from 4/7 to 7/7 modules gains +2.5 pp on no-prefix C2 but loses -2.0 pp on cmaj. Trainable parameters rise $10\text{M} \rightarrow 22\text{M}$ ($2.2\times$). Format-robustness on the static-prefix axis and sampling-diversity-on-vote on the branch-vote axis are both affected by Track-1-style training, but in opposite directions at higher capacity. We interpret this as additional capacity drifting the model further from base on the high-confidence answer-token path that vote-aggregation relies on; the cmaj ensemble is more sensitive to per-branch drift than the single-shot reading is.

(b) **The planner-quality split inverts.** At v2 capacity the bigger QWEN2.5-1.5B planner produces better plans and the adapter uses them (+7 pp). At v3 capacity, the same bigger planner causes a -13 pp regression while the smaller QWEN2.5-0.5B planner improves +5 pp. This is the seed for §4: capacity does not just flatten format brittleness, it modulates content trust, and the direction of the effect is opposite for the two planners.

3.4 Reading

Track 1 establishes that interface-format brittleness is a real, isolable, trainable axis. An $r=8$ LoRA on a small prefix-randomized dataset closes the static-prefix damage gap from 8 pp to within 1 pp at v3 capacity. The cost is split between -2.0 pp on cmaj (capacity tradeoff) and a widened spread across content-rich plans (motivating §4). For deployment, the v2-vs-v3 choice depends on whether the downstream pipeline uses single-shot (v3 wins by +2.5 pp on C2) or branch-vote (v2 wins by +2.0 pp on cmaj).

3.5 A Note on the v1 Recipe Catastrophe

Both v2 and v3 above followed a recipe revision motivated by a straightforward, publishable failure of the first attempt (v1) at this same training objective. v1 used $r=16$, $\alpha=32$, learning

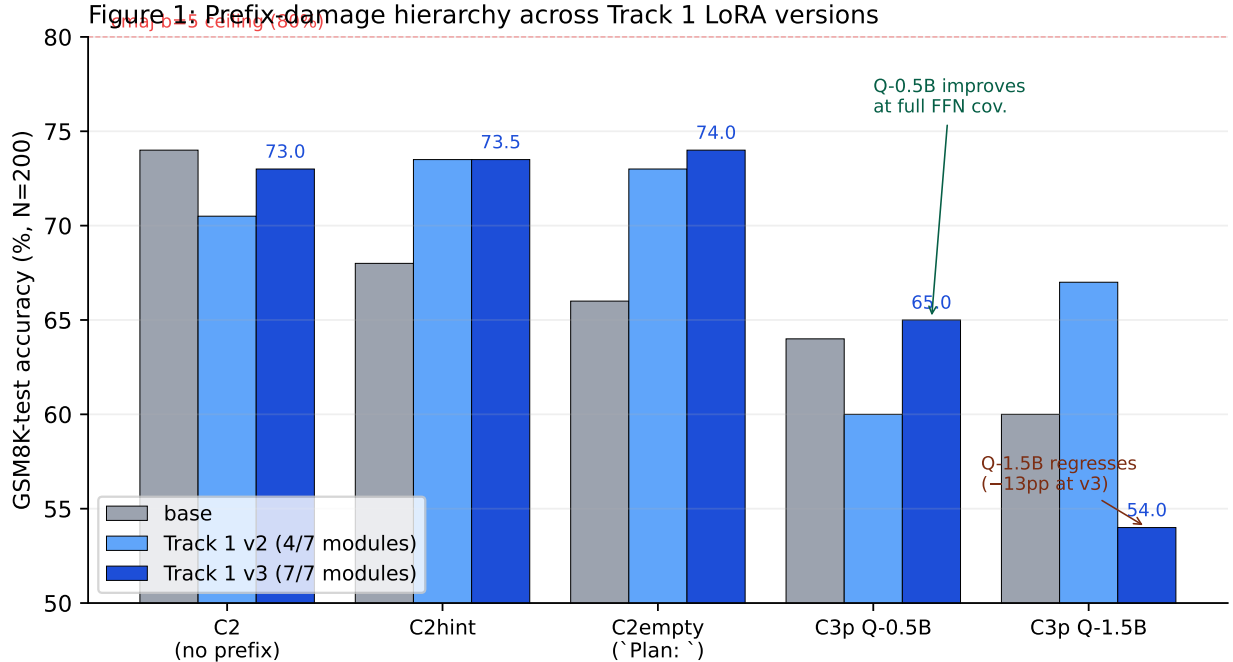


Figure 1: Prefix-damage hierarchy across $\{\text{base, Track 1 v2, Track 1 v3}\}$ on all five conditions. The QWEN2.5-0.5B/QWEN2.5-1.5B inversion at v3 is annotated. Static-prefix damage flattens at v2 and tightens further at v3; planner-content sensitivity widens at v3.

rate $1e-4$, mask-sampling $p_{\text{mask}} \sim U(\epsilon, 1-\epsilon)$, and 14k training steps. The resulting adapter regressed C2 from 74% to 59.5% (-14.5 pp) via a clean bigram-repetition mode collapse: at inference, the denoised CoT block degenerated into copy-back-context loops (e.g. "He sold the house for 50000+80000=\$<<50000+80000=1He sold the house for 50000+80000=\$<<50000+80000=130000>>130,000", with the prefix phrase repeated verbatim). v2 fixed this with four hyperparameter changes from a single design lesson: the mask-sampling distribution $p_{\text{mask}} \sim U(\epsilon, 1-\epsilon)$ over-weights the high-mask cases, training the model to predict tokens from very little context, which in turn corrupts low-mask inference — the regime GSM8K reasoning actually exercises. Narrowing the support to $U(0.1, 0.5)$, dropping the LoRA rank to $r=8$, and reducing the LR and step count yielded the v2/v3 recipes reported above.

We surface this in the body rather than burying it in Appendix C because it is itself a publishable masked-diffusion SFT pitfall: a recipe that looks fine on validation loss can produce an adapter that catastrophes at the inference distribution due to a mismatched mask-sampling prior. The contrast between v1 (-14.5 pp) and v2 ($+0$ pp on C2 but lifts on every prefixed condition) is a generally useful warning for masked-diffusion adapter recipes elsewhere.

Status. Track 1’s prefix-robustness fix is robust on GSM8K-test $N=200$, single seed, with the prefix-tier distribution described in §2. Cross-task transfer (e.g. MATH-Easy, ARC-Challenge) and multi-seed variance are open and flagged in Appendix E; both are cheap follow-ups that would harden the headline numbers without changing the framing.

4 Phenomenon 2: Planner-Content Trust Is Capacity-Dependent in Opposite Directions Across Planners

The cleanest signal in the v2-vs-v3 comparison (Table 3) is the inversion of the QWEN2.5-0.5B-vs-QWEN2.5-1.5B planner split. At v2 capacity (4/7 modules, 10M parameters), the larger planner helps and the smaller one hurts: C3p Q-1.5B reaches 67.0% versus C3p Q-0.5B at 60.0% ($\Delta = +7$ pp). At v3 capacity (7/7 modules, 22M parameters), the directions invert: C3p Q-1.5B drops to 54.0% while C3p Q-0.5B rises to 65.0% ($\Delta = -11$ pp). This is a previously-unmeasured *phenomenon*: LoRA capacity differentially affects how much the model *trusts* the content of an up-stream plan, and the direction of the effect is opposite for the two planners. We have two capacity rungs (v2 at 10M and v3 at 22M trainable parameters); the data establishes that the inversion is real, but it does not yet trace a fitted non-monotonic curve. We label this a phenomenon rather than an axis on the same footing as brittleness (§3) and diversity (§5) for that reason; a third capacity rung (e.g. 5M or 50M) and per-planner multi-seed runs would upgrade it.

4.1 The Pattern

Table 4 extracts the planner conditions from §3 and adds the v2 $\rightarrow$ v3 deltas.

Table 4: Planner-content trust as a function of LoRA capacity. At v2 the bigger planner helps; at v3 it catastrophically regresses while the smaller planner improves. Static-prefix conditions (C2hint, C2empty) are unaffected by the capacity bump and are listed for contrast.

Condition	Base	v2	v3	v2 $\rightarrow$ v3
C2hint (static)	68%	73.5%	73.5%	0.0
C2empty (static)	66%	73.0%	74.0%	+1.0
C3p Q-0.5B (content)	64%	60.0%	65.0%	+5.0
C3p Q-1.5B (content)	60%	67.0%	54.0%	-13.0

The capacity bump is invisible on the static-prefix conditions (both within 1 pp of v2) and amplifies in opposite directions on the content-rich conditions. Since the only thing that distinguishes C3p Q-0.5B from C3p Q-1.5B is the content of the prefix — the surface shape is identical — the capacity-amplified differential is an effect on plan-content processing, not on plan shape.

4.2 Reading

The natural interpretation: full-FFN coverage at v3 makes the adapter *more* sensitive to the content of the prefix. For QWEN2.5-0.5B, the prefixes are simple and often noisy, and the extra sensitivity is corrective — the LoRA learns to ignore weak content effectively, recovering +5 pp. For QWEN2.5-1.5B, the prefixes are more sophisticated and contain plan-internal patterns that the adapter latches onto. The increased sensitivity makes the model overfit to those patterns at training time and they do not generalize at evaluation, regressing past the base 60% baseline by -6 pp net.

This reading is consistent with the static-prefix conditions being unaffected by the capacity bump: "Plan: " and "Let's think step by step." have no content for the adapter to over-fit to, so additional capacity has no per-content attractor to drift toward. Only the planner conditions, where the prefix carries question-specific content, expose the trust modulation.

For deployment, the practical implication is that the v2/v3 choice is not a uniform Pareto frontier: it depends on the size of the upstream planner. A pipeline that pairs the prefix-robustness LoRA with a $\sim 0.5\text{B}$ planner should ship v3; a pipeline that pairs it with a $\sim 1.5\text{B}$ planner should ship v2. We did not test QWEN2.5-7B planners; the trust modulation may continue or it may bend back at larger planner scale, and characterizing the full curve is left to future work.

Status. This paper characterizes the axis but does not fix it. A content-trust adapter — LoRA training that varies plan content quality at fixed surface shape, rather than varying surface shape at fixed content as our prefix-robust dataset does — is a natural follow-up. We sketch the design in §11.

5 Axis 3: Sampling Diversity Is *Expanded* by Format-Augmented Training

The third axis is the one we expected to be the easiest call and were wrong about. Pre-registration prediction #3 in e2/PROTOCOL.md predicted that branch-vote diversity would be *preserved* after Track 1 training: the prefix-robust LoRA should not collapse the stochastic sampler that cmaj relies on, and cmaj accuracy should stay within ± 1 pp of base. We falsify this prediction in the unexpected direction: diversity *expands*, and cmaj accuracy is preserved or slightly improved.

5.1 Branch-Agreement Distribution

We measure the per-problem distribution of unique-answer counts across 5 stochastic branches at $t=0.7$, on the same $N=200$ GSM8K-test eval used elsewhere. Table 5 reports the distribution for base LLADA and for Track 1 v2 + commit v2.

Table 5: Branch-agreement distribution on GSM8K-test, $N=200$, $b=5$, $t=0.7$. Apples-to-apples: both columns evaluated on the same 200 test problems. Mean unique answers per problem rises $1.83 \rightarrow 2.07$ (+13%); the fraction of problems where all 5 branches agree drops from 51.5% to 47.5%. cmajc accuracy on test is 82.0% (commit v2) / 82.5% (commit v3) — diversity goes up while branch-vote accuracy is preserved or improved (Table 6).

Bin	Base on test	Track 1 v2 + commit v2
5/5 same answer	51.5%	47.5%
4/5 unique answers	6.0%	8.5%
3/5 unique answers	11.5%	18.0%
2/5 unique answers	27.5%	19.5%
5/5 unique answers	3.5%	6.5%
Mean unique answers / problem	1.825	2.07

95% CIs for bin fractions (Clopper–Pearson, $N=200$, base / Track 1 v2 + commit v2): 5/5 same [44.3, 58.6] / [40.4, 54.7]; 4/5 unique [3.1, 10.2] / [5.0, 13.3]; 3/5 unique [7.4, 16.8] / [12.9, 24.0]; 2/5 unique [21.4, 34.2] / [14.2, 25.7]; 5/5 unique [1.4, 7.1] / [3.5, 10.9]. The mean unique-answers row is a per-problem mean of a count in $\{1, \dots, 5\}$; a paired bootstrap CI for the $\Delta = +0.245$ shift requires per-problem unique-counts and is supplied with the released per-problem JSONLs.

The shift is consistent across bins: less concentration on 5/5 agreement, more mass at 3/5 and 5/5 unique. Mean unique answers per problem rises by +13%. And branch-vote accuracy *still*

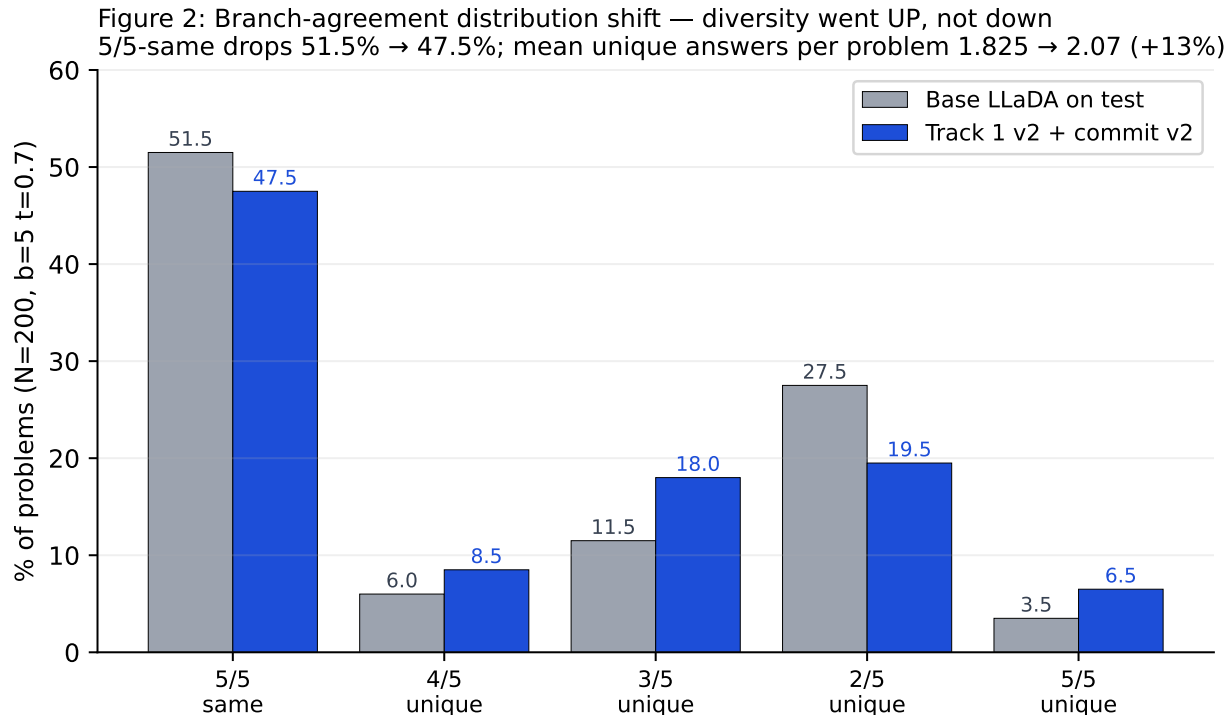


Figure 2: Branch-agreement distribution shift, 5-bin histogram for {base on test, Track 1 v2 + commit v2}. The drop in 5/5-same and the rises in 3/5-unique and 5/5-unique are visible. Generated by `scripts/make_paper_figures.py`.

holds at 81.5% on dev200 (Track 1 v2, Table 2) and lifts on test under `cmajc`: 82.0% with commit v2 and 82.5% with commit v3 (+3 / +3.5 pp over base test `cmaj` of 79.0%, see Table 6). Diversity goes up and accuracy is preserved or improved.

5.2 Hypothesis: Surface-to-Trajectory Generalization

Format-augmented training (§2, eight prefix tiers per problem) implicitly teaches the LoRA “many surfaces, same content”. A single GSM8K problem appears in the training set under eight different prefix variants but with the same target response. We hypothesize that this lesson generalizes at inference into “many CoT trajectories, same answer”: the adapter not only preserves sampling variance, it teaches the model to explore more.

Phrased mechanistically, the eight-tier surface-randomization acts as an implicit data augmentation that flattens high-confidence attractor basins on the prefix-shape axis, spreading probability mass across nearby basins on the trajectory axis when the sampler is run at $t=0.7$. This is the opposite of the standard encoder-collapse story, where training folds representations into fewer attractors. We observe expansion, not collapse, of the inference-time attractor distribution.

This finding is independent of the v2-vs-v3 capacity tradeoff (§3): we report the diversity-expansion result on v2 + commit v2 because that pairing has both the longest eval record and the matching test-set base `cmaj`. The corresponding `cmajc` number for v3 + commit v3 (82.5%, statistically indistinguishable from v2’s 82.0% at $N=200$) is reported in Table 6.

5.3 Open Mechanism Question

Whether the diversity uptick is an *implicit-temperature* effect (raising the base sampler to $t=0.9$ would reproduce the 47.5% 5/5-same rate) or a real *content-diversity* effect (different reasoning paths, not just different surface jitter) is open. The cheap test is to evaluate base LLADA at $t \in \{0.8, 0.9, 1.0\}$ on the same eval and read the 5/5-same rate. We deferred this experiment to revision; the result does not change the headline that diversity is expanded rather than collapsed, but it changes the mechanism story considerably.

Reading. The diversity-expansion result is the second pre-registration violation in this paper after the cmajc-vs-cmaj compositionality result (§6). Both violations are in directions favorable to the hybrid pipeline: more diversity than expected, more compositional gain than expected. We treat them as positive surprises and document them as falsified predictions rather than as findings to be celebrated.

6 Consensus Distillation Is Design-Sensitive, Not Architecture-Limited

The motivating gap for Track 2: cmaj $b=5$ reaches 80% on dev (and 79% on test, apples-to-apples) while greedy single-shot C2 reaches 74%. A 5–6 pp gap that scales linearly in b up to $b=5$ (saturated). Track 2’s question: can a small commit-adapter LoRA close this gap by distilling cmaj’s consensus into a single forward pass?

This section reports two design iterations that fail ($c2c = 70.5\%$ across $3.25\times$ capacity), a third design that succeeds ($c2c = 79.0\%$, within sampling error of the 80% target), and disentangling ablations that attribute the lift to where commit fires in the diffusion schedule rather than to the supervision signal. The headline is that the v1/v2 plateau is a *design* bottleneck, not an architectural impossibility, and the two design errors mask each other.

6.1 The Plateau: v1 and v2 at $c2c = 70.5\%$

The training data is `eren23/sfumato-commit-mixture-gsm8k` — 109 rows derived from running cmaj on 500 GSM8K-train problems and keeping three buckets (46 rescue, 58 preserve-disagreement, 5 pure-agreement; see §2).

v1 uses target modules [`gate_proj`, `up_proj`, `down_proj`]. Of these only `up_proj` matches LLADA’s actual block, so the LoRA attaches to 1/3 of FFN, with 4.2M trainable parameters. $c2c = 70.5\%$.

v2 fixes the module-name bug: target modules [`ff_proj`, `up_proj`, `ff_out`] — the LLADA-correct list, 3/3 FFN match, 13.6M trainable parameters ($3.25\times$ v1). Same data, same hyperparameters, same answer-span loss, same last-block-only inference. $c2c = 70.5\%$.

The same 70.5% across a $3.25\times$ capacity bump rules out a capacity-limited reading: whatever is bottlenecking the adapter is not parameter count. A diagnostic comparison of decoded text with `apply_commit=True` versus `False` at $t=0$ (deterministic) on 10 fixed problems confirmed this: text shifts in 8 of 10 problems, but the answer never flips. The commit LoRA *was* modifying tokens, but only late-block tokens, which could not change the answer that earlier blocks had already pinned.

6.2 v3: Multi-Block Commit + Full-Response Loss

Two design changes for v3, on the same $r=8$ FFN-only LoRA and the same dataset:

1. **Multi-block commit at inference:** enable the commit adapter on sub-blocks 2–4 of 4 (the trailing 75% of the 128-token generation), not just the last sub-block. This corresponds to `COMMIT_N_BLOCKS=3` in the inference script.
2. **Full-response training loss:** the masked-diffusion CE loss is computed over the entire response span `[prompt_len, response_end]` rather than the answer-tail span `[answer_start, response_end]`. This corresponds to the `FULL_RESPONSE_LOSS=1` flag in the trainer.

Both changes are inexpensive: the trainer adds a few lines, and the inference change is a single integer flag.

Table 6 reports the headline numbers.

Table 6: Track 2 commit-adapter results on GSM8K-test, $N=200$, $k=64$, seed 0. v3 c2c = 79.0% is within sampling error of the 80% pre-registered target (binomial CI ± 5 pp at $N=200$). cmajc v2 = 82.0% on test is +3 pp over base test cmaj = 79.0%.

Setup	c2c	cmajc
Track 1 v2 alone (no commit)	70.5%	—
+ commit v1 (last-block, answer-span)	70.5%	81.5%
+ commit v2 (last-block, answer-span)	70.5%	82.0%
Track 1 v3 alone (no commit)	73.0%	—
+ commit v3 ($n=3$, full-response)	79.0%	82.5%

95% CIs (Clopper–Pearson, $N=200$): Track 1 v2 alone c2c [63.7, 76.7]; +commit v1 c2c [63.7, 76.7], cmajc [75.4, 86.6]; +commit v2 c2c [63.7, 76.7], cmajc [76.0, 87.1]; Track 1 v3 alone c2c [66.3, 79.0]; +commit v3 c2c [72.7, 84.4], cmajc [76.5, 87.5].

At $N=200$, the exact binomial 95% CI for v3 c2c is 79.0% [72.7, 84.4], which contains the 80% pre-registered target. Track 2’s pre-registered prediction #1 (c2c $\geq 80\%$) is therefore within CI of holding, with the point estimate +8.5 pp over the v1/v2 plateau.

6.3 Disentangling: Block Coverage Versus Supervision

The v3 result bundles three changes from v2: full-FFN module coverage on the upstream Track 1 LoRA (v2 $\rightarrow$ v3, +2.5 pp on no-prefix C2), n_{blocks} 1 $\rightarrow$ 3 on inference, and answer-span $\rightarrow$ full-response on training. We disentangle the second two with two ablations on the same Track 1 v3 base, holding everything else fixed.

Block coverage ($n_{\text{blocks}}=3$) is the dominant driver: +4.0 pp on its own, even with the sub-optimal v2 answer-span supervision (ABL_A). **Full-response loss alone:** +0.0 pp (ABL_B). Full-response training has *no leverage* if commit application is still gated to the last sub-block. **Combination:** +6.0 pp. Full-response loss adds +2.0 pp on top of multi-block commit.

The two design errors mask each other. Late-block-only commit is a structural bottleneck that prevents the supervision change from doing any work, and answer-span-only training is a supervision bottleneck that prevents the structural change from doing all of its work. The cleanest one-sentence statement: *once commit fires earlier in the diffusion schedule, even sub-optimal supervision recovers most of the value.*

The **ABL_B** null result is consistent with the mechanism story we develop in §6.4; a sanity probe verifying the v3 adapter loads and modifies inference at the `COMMIT_N_BLOCKS=1` configuration is deferred (Appendix E).

Table 7: Disentangling ablations. Block coverage ($n_{\text{blocks}}=3$) drives most of the lift. Full-response loss adds +2.0 pp on top, but on its own (with $n_{\text{blocks}}=1$) adds nothing. The two design errors mask each other.

Setup	LoRA	commit	n_{blocks}	training loss	c2c / Δ
Track 1 v3 alone	v3	—	—	—	73.0% / —
ABL_A	v3	v2	3	answer-span	77.0% / +4.0
ABL_B	v3	v3	1	full-response	73.0% / +0.0
v3 full	v3	v3	3	full-response	79.0% / +6.0

95% CIs (Clopper–Pearson, $N=200$): Track 1 v3 alone [66.3, 79.0]; **ABL_A** [70.5, 82.6]; **ABL_B** [66.3, 79.0]; **v3 full** [72.7, 84.4]. CIs on the **ABL_A** vs. **v3 full** difference (+2 pp on top of multi-block commit) and the **ABL_B** vs. Track 1 v3 alone non-difference are paired-sample quantities; per-problem JSONLs released alongside the adapters support paired analysis.

6.4 Mechanism

The 96-token prefix problem. LLADA’s semi-AR sampler generates 128 tokens in 4 sub-blocks of 32. By the time the sampler reaches the last sub-block, the previous 96 tokens have committed the chain-of-thought, and the answer is largely implied by the reasoning that has already been laid down. A commit-LoRA that fires *only* on the last sub-block can shift the literal answer-tokens, but those tokens are downstream of the reasoning that already determined the answer. The diagnostic 8/10 text-shift, 0/10 answer-flip result (§6.1) is exactly this picture.

When commit fires on sub-blocks 2–4, the LoRA can shape the *trajectory* of the late CoT, not just the answer literal. Once the trajectory is malleable, the answer-token shift becomes load-bearing because the answer-tokens are now downstream of a trajectory the adapter helped produce.

Full-response training reinforces this. An answer-span-only loss teaches the LoRA to predict the answer given a fixed CoT context, which is approximately what the base model already does. A full-response loss teaches the LoRA to predict the entire trajectory of consensus-quality reasoning, which is the harder task and the one that generalizes when commit is applied to mid-trajectory blocks. The **ABL_B** null result (Table 7, +0.0 pp from full-response training when commit fires only on the last sub-block) is the cleanest empirical signature of this mechanism: even better supervision cannot rescue surgery whose temporal placement is wrong.

6.5 Compositionality with Branch Aggregation

Pre-registered prediction #3 was that **cmajc** (**cmaj** with commit-LoRA per branch) would not exceed **cmaj** $b=5$ by more than 1 pp — the intuition being that if commit-LoRA is doing useful work, that work should already be captured by majority vote across stochastic branches, and stacking should produce no double-dip.

The result with commit v2: **cmajc** = 82.0% [76.0, 87.1] on test versus base test **cmaj** = 79.0% [72.7, 84.4], a +3 pp delta. The result with commit v3 (the c2c-fixed design, **COMMIT_N_BLOCKS=3**, full-response training): **cmajc** = 82.5% [76.5, 87.5], a +3.5 pp delta over base test **cmaj**. Pre-reg violated upward in both cases. Branch aggregation and weight-space commit do partially independent work, and the gains compose under both Track 2 designs.

Branch aggregation absorbs much of the structural correction that the v3 commit delivers in single-pass inference. Commit v3 beats commit v2 by +8.5 pp on the single-pass c2c setting (79.0% vs 70.5%) but matches it statistically on the branch-aggregated **cmajc** setting

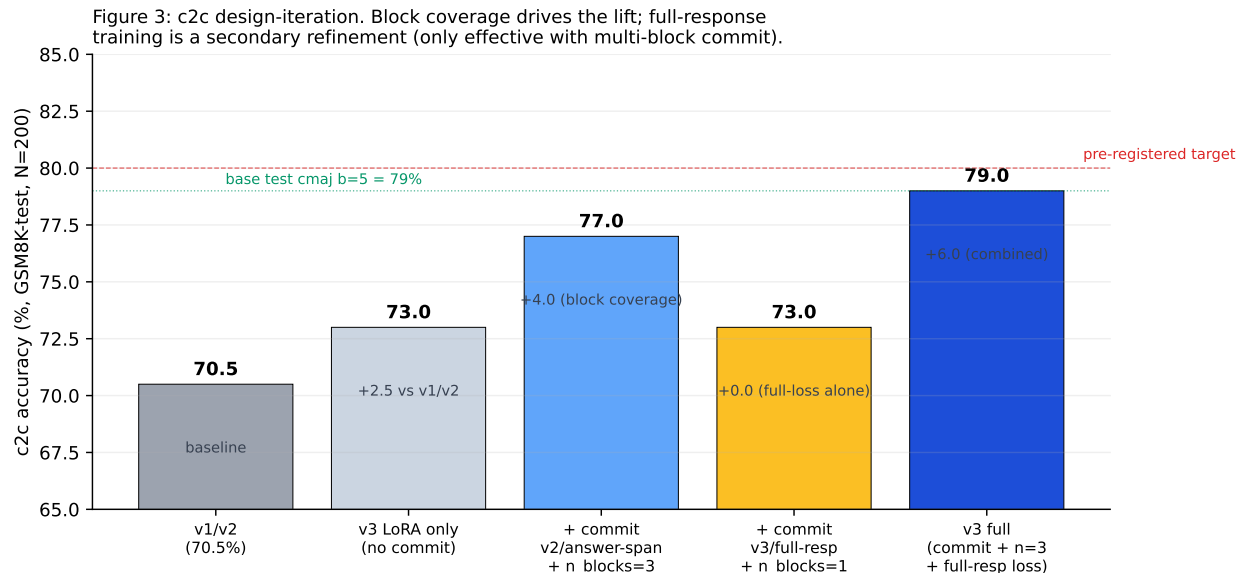


Figure 3: c2c design-iteration disentangling. Five bars: v1/v2 baseline, v3 LoRA alone, ABL_A (block coverage with v2 answer-span supervision), ABL_B (full-response loss with last-block-only commit), v3 full. The pre-registered 80% target and base test `cmaj` of 79% lines are visible. The +0.0 result for ABL_B (full-response loss with $n_{\text{blocks}}=1$) makes the “block coverage is dominant” story visually unambiguous. Generated by `scripts/make_paper_figures.py`.

(82.5% vs 82.0%, both within each other’s CI at $N=200$). Figure 4 visualizes the collapse.

The mechanistic reading: `cmaj` selects among trajectories sampled at temperature; commit-LoRA biases each individual trajectory toward consensus-quality reasoning before voting. At $b=5$, branch aggregation already recovers much of the consensus quality that the late-block-only commit (v2) misses, so additional structural correction at the per-trajectory level (v3) has less incremental room to operate. The implication for distillation practice: where the v3 commit’s marginal contribution is largest is exactly where it is most needed — at $b=1$. For single-pass inference, the structurally-correct design pays; under stochastic-aggregation regimes, the structurally-incorrect design is indistinguishable.

Status. `cmajc` on the v3 commit was added in this revision and reaches 82.5% [76.5, 87.5]; the `cmajc-v2` number (82.0%) remains in Table 6 for the historical record. The two are statistically indistinguishable at $N=200$.

7 Reviewer-Resilience: Is This Just Self-Consistency?

The most natural alternative explanation for everything reported above is generic AR self-consistency [18]: sample b branches at temperature t , take the majority vote, and get the same accuracy lift the diffusion `cmaj` mechanism appears to deliver. If true, the diffusion-specific story — that LLADA’s within-block uncertainty produces genuinely different reasoning paths rather than surface jitter — would be unnecessary.

We rule this out empirically with a matched baseline.

Figure 4: Branch aggregation absorbs the v2→v3 structural correction at $b=5$.

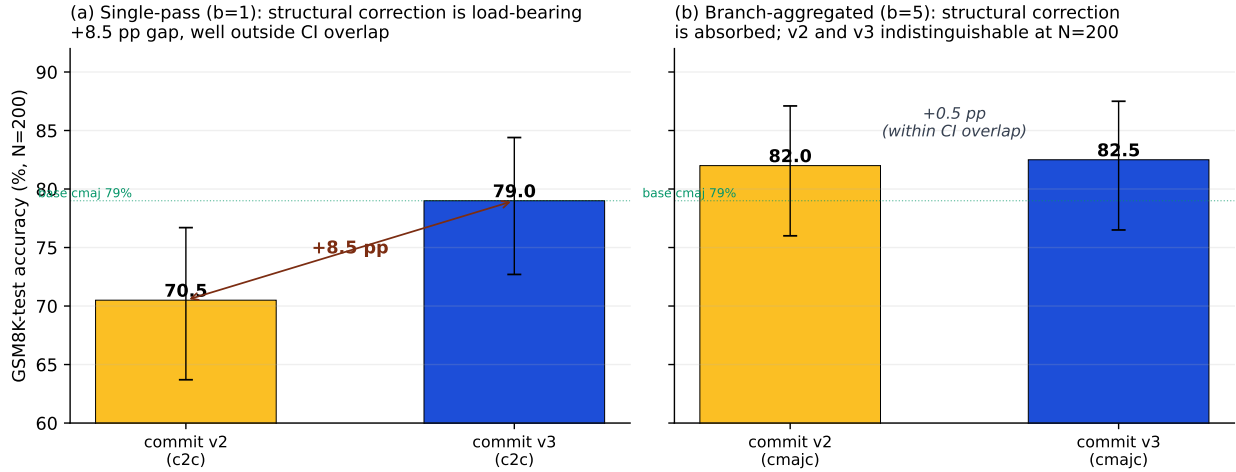


Figure 4: Branch aggregation absorbs the v2→v3 structural correction. At $b=1$ (left), the structurally-correct v3 commit beats the structurally-incorrect v2 commit by +8.5 pp, well outside CI overlap. At $b=5$ (right), the same two adapters are statistically indistinguishable. Stochastic branch-vote already recovers the consensus quality that the late-block-only v2 commit misses, leaving the v3 commit’s structural advantage with little incremental room to operate. Generated by `scripts/make_paper_figures.py`.

7.1 Baseline

QWEN2.5-0.5B-Instruct [17] self-consistency at $b=5$, $t=0.7$, on the same GSM8K-test eval ($N=200$, fixed problem indices, fixed seed):

Table 8: AR self-consistency baseline against LLADA `cmaj` on the same eval ($N=200$, GSM8K-test, $b=5$, $t=0.7$). Generic self-consistency does not explain the diffusion `cmaj` advantage.

Setup	Accuracy
QWEN2.5-0.5B self-consistency, $b=5$, $t=0.7$	40.5%
LLADA <code>cmaj</code> on test (base, no LoRA)	79.0%
LLADA <code>cmaj</code> (Track 1 v2 + commit v2)	82.0%

The gap between QWEN2.5-0.5B self-consistency and LLADA `cmaj` is 38.5–41.5 pp. Generic self-consistency at matched b , t , and eval does not explain the advantage.

7.2 Reading

The QWEN2.5-0.5B planner is the same model used for the C3p Q-0.5B condition elsewhere in this paper. At that scale, AR self-consistency does not produce enough genuinely different reasoning paths to vote effectively: most of the 5 branches differ in surface phrasing but not in the answer. LLADA’s diffusion sampler resolves within-block uncertainty jointly across many tokens, producing trajectories that diverge in their *reasoning* rather than only in their phrasing. This is consistent with the diversity-expansion finding in §5: the diffusion sampler at $t=0.7$ is already exploring

enough trajectory variety that **cmaj** has real disagreement to aggregate over, and Track 1 training expands the distribution further rather than collapsing it.

This rebuttal is narrow. It does not show that diffusion **cmaj** beats AR self-consistency at all scales — a 7B QWEN2.5- planner with self-consistency would likely close some of the gap. It shows that at the matched-base-model scale relevant to this paper, the simplest alternative explanation is empirically inadequate.

8 The Voting-Rule Gap and Verifier Aggregation

The previous sections rule out generic AR self-consistency as the explanation for LLADA’s **cmaj** advantage and characterize three axes of trainable hybrid failure. A different question — and the single most actionable observation in our follow-up phase — is whether **cmaj** itself is leaving accuracy on the table. It is. Across every **cmaj** configuration evaluated, the *oracle* ceiling (mark a problem solved if any of the $b=5$ branches is correct) exceeds the **cmaj** majority vote by 8–12 pp. This section reports that gap, two negative results on closing it with self-trained per-branch verifiers, the encoder-scaling trend that survives those negatives, and a positive control with a frontier-class judge that closes 86% of the gap. The mechanism that ties the four findings together is that LLADA’s failure mode on GSM8K is problem-comprehension, not arithmetic.

8.1 The Voting-Rule Gap is Structural

Table 9: Oracle ceiling minus **cmaj** accuracy across four configurations and three seeds, GSM8K-test $b=5$, $t=0.7$, $k=64$. Square brackets are exact (Clopper–Pearson) binomial 95% CIs. “Oracle” marks a problem solved if at least one of the five branches is correct. The gap is 8–12 pp and is roughly invariant to LoRA upgrades.

Configuration	N	cmaj a_b	Oracle	Gap
Base LLADA-8B-Instruct, $\tau=0.7$	50	78.0%	90.0%	12.0 pp
Track-1 v3 LoRA, $\tau=0.7$	200	79.5% [73.3, 84.9]	88.0% [82.7, 92.2]	8.5 pp
v3 LoRA + commit-v3, seed 1, $\tau=0.7$	100	82.0%	90.0%	8.0 pp
v3 LoRA + commit-v3, seed 2, $\tau=0.7$	100	82.0%	90.0%	8.0 pp

In the median problem the right answer is present in at least one of the five sampled branches; majority voting discards it. The gap holds across the base model, the v3 prefix-robust LoRA, and the v3+commit-v3 stack, suggesting that the residual error is a structural property of **cmaj** aggregation rather than a model-quality artifact addressable by further LoRA training.

8.2 Multi-Seed Variance Bar

Seeds 1 and 2 in Table 9 were also run for the same purpose as Phase-1 paper-hardening: to rule out that the 82.5% **cmajc** headline (seed 0, $N=200$, Track-1-v3 + commit-v3) was a seed-dependent artifact. Three-seed mean **82.2%**, sample standard deviation **0.29 pp** — tighter than the Wilson interval at $N=100$. The Phase-1 **cmajc** headline is robust to seed selection; this closes the multi-seed variance exposure originally listed in Appendix E.

8.3 Per-Branch Verifiers: Seventeen Architectures, All Lose

The 8–12 pp gap is enough to motivate a per-branch verifier that reranks the five `cmaj` branches at inference time. We pre-registered a success threshold of $\geq 83\%$ mean accuracy on 5-fold cross-validation split by `problem_id` (to prevent leakage on the same problem appearing in train and eval folds). Across 17 architectures spanning four orders of magnitude in encoder size and four distinct architecture families — text-bag classifiers, chat-LM mean-pool encoders, embedding-specific models, math-tuned chat models, process-feature MLPs, contrastive branch-pair models, step-level PRMs, and a symbolic-arithmetic checker — *every* architecture underperformed the `cmaj` baseline. Table 10 summarizes the top of the leaderboard; the full ranking (23 entries) is released as `phase2/RANKING.md` alongside the per-spike result JSONs.

Table 10: Top of the verifier leaderboard, GSM8K-test $b=5$, $t=0.7$ `cmaj` substrate. All peer-class verifiers lose to `cmaj`; only a frontier-class judge with chain-of-thought wins (§8.6).

Approach	<code>cmaj</code>	Verifier	Oracle	Δ_{pp}	Decision
Claude Sonnet 4.5 + CoT	79.1%	85.3%	86.3%	+6.16	WIN-MINOR
Claude Sonnet 4.5 (YES/NO)	79.1%	82.9%	86.3%	+3.79	inconclusive
DeepSeek-R1-Distill-Qwen-32B	80.5%	77.5%	89.5%	−3.00	LOSS
Qwen2.5-32B-Instruct (4-bit)	80.5%	77.0%	89.5%	−3.50	LOSS
Qwen2.5-7B-Instruct + MLP	80.5%	76.5%	89.5%	−4.00	LOSS
Process-MLP ($N=100$ rich substrate)	85.0%	81.0%	91.0%	−4.00	LOSS
Qwen3-8B chat + MLP	80.5%	75.0%	89.5%	−5.50	LOSS
Qwen2.5-Math-7B + MLP	80.5%	74.0%	89.5%	−6.50	LOSS
Qwen3-Embedding-8B + MLP	80.5%	72.5%	89.5%	−8.00	LOSS
Qwen2.5-0.5B + MLP	80.5%	72.0%	89.5%	−8.50	LOSS
Symbolic arithmetic checker	79.1%	70.1%	86.3%	−8.98	LOSS
TF-IDF + logistic regression	80.5%	66.5%	89.5%	−14.0	LOSS

8.4 Three Robust Negative Findings

Across the leaderboard three patterns are stable enough to call out as findings on their own.

Math-tuning hurts as a verifier. Qwen2.5-Math-7B and Qwen2.5-Math-RM-72B both underperform their non-math-tuned counterparts at the same parameter count (−6.5 pp vs −4.0 pp for Qwen2.5-7B chat). Math-specific fine-tuning appears to compress features around math *vocabulary* (“is this math?”), losing the discrimination needed to rank sibling-but-different math-CoT branches by correctness (“is this math right?”).

Embedding-specific models lose worst. BGE-M3, Qwen3-Embedding-4B/8B, and gte-Qwen2-7B all underperform chat-LMs of the same size as feature extractors. Qwen3-Embedding-8B (−8 pp) is dominated by the *smaller* Qwen3-8B chat (−5.5 pp). Cosine/InfoNCE training compresses embeddings around *similarity*, which is the wrong invariant: branches differing only in problem setup are similar by construction.

Doubling the substrate makes verifiers worse. Holding architecture fixed and increasing N from 200 to 500 moves Δ_{pp} in the wrong direction — $-5.5 \rightarrow -6.16$ for the process-MLP and -8.98 for the symbolic checker (vs −8 pp at $N=200$). This is the signature of an overfit-to-substrate-noise regime, and is consistent with the 8–12 pp gap being structural rather than data-bound.

8.5 Encoder Scaling is Monotone but Insufficient

Within the chat-LM mean-pool + MLP family, scaling the encoder *monotonically* narrows the gap (Table 11).

Table 11: Verifier accuracy versus encoder scale, 5-fold CV by `problem_id` on the 1750-branch `cmaj` substrate. Each $\sim 10\times$ scale-up of the encoder narrows the gap to `cmaj` by ~ 5 pp on average; linear extrapolation is consistent with a 32B–72B encoder crossing the `cmaj` baseline, with the caveat that the curve may flatten further.

Encoder	Params	Mean verifier	Δ_{pp} vs <code>cmaj</code>	Gap closure
TF-IDF + LR	$\sim 250\text{K}$	66.5%	−14.0	−156%
Qwen2.5-0.5B-Instruct + MLP	500M	72.0%	−8.5	−94%
Qwen2.5-7B-Instruct + MLP	7.6B	76.5%	−4.0	−44%

We label the trend rather than the extrapolation as the publishable observation. The reachable accuracy *within* the chat-LM-mean-pool + MLP family at 0.5B and 7B scale is strictly below `cmaj` on this substrate; whether the trend holds at 32B+ is a Phase-3 question.

8.6 A Frontier Judge with CoT Closes 86% of the Gap

The negative results above are bounded by an explicit positive control. We call Claude Sonnet 4.5 [4] through the OpenRouter API [16] once per branch, under two prompts: a YES/NO verdict prompt and a chain-of-thought-then-verdict prompt that forces the judge to reconstruct the problem setup before judging the candidate solution. Substrate is the GSM8K-test $N=500$ extension (200 frozen indices + 300 fresh, $b=5$, $t=0.7$, Track-1-v3 LoRA).

Table 12: Frontier judge result, GSM8K-test $N=500$, $b=5$, $t=0.7$, Track-1-v3 substrate. CoT prompting recovers an additional +2.4 pp over the YES/NO prompt by forcing problem reconstruction.

Approach	<code>cmaj</code>	Verifier	Oracle	Δ_{pp}	Gap closure
Claude Sonnet 4.5 + CoT	79.1%	85.3%	86.3%	+6.16	86%
Claude Sonnet 4.5 (YES/NO)	79.1%	82.9%	86.3%	+3.79	53%

Mechanism. Hand inspection of the 15 `cmaj`-failed/oracle-recoverable problems on the $N=500$ substrate reveals that wrong branches typically contain 100% correct arithmetic but apply it to incorrect problem setups. Figure 5 illustrates the failure mode on a representative problem (GSM8K-test problem 1071). All five branches do correct arithmetic; three branches misread “\$6 for each *additional* guest” as “\$6 for each guest” and arrive at a self-consistent \$371; two branches read the constraint correctly and arrive at \$251. Majority vote picks the wrong setup; the oracle ceiling and a frontier judge that re-derives the constraint both recover the correct answer.

This single observation predicts the entire leaderboard:

1. **Symbolic verifiers lose** (−8.98 pp): every branch’s arithmetic statement-checks pass equally, because the arithmetic is correct everywhere.

Question (GSM8K-test #1071). Kayla is having her birthday party at a movie theater. *The fee to rent the theater is \$125 for a party of 20, plus \$6 for each **additional** guest.* Kayla invited her 25 classmates and the 7 girls in her dance class, as well as 13 family members. Only 4 people said they could not come. How much will the party cost? *Gold answer: \$251.*

Branch	Setup (charge \$6 to ...)	Arithmetic	Answer	Outcome
0	all 41 attending guests	$125 + 41 \cdot 6 = 371$	\$371	<i>wrong</i>
1	$41 - 20 = 21$ additional	$125 + 21 \cdot 6 = 251$	\$251	<i>correct</i>
2	$41 - 20 = 21$ additional	$125 + 21 \cdot 6 = 251$	\$251	<i>correct</i>
3	all 41 attending guests	$125 + 41 \cdot 6 = 371$	\$371	<i>wrong</i>
4	all 41 attending guests	$125 + 41 \cdot 6 = 371$	\$371	<i>wrong</i>
cmaj majority vote (3/5)			\$371	<i>(wrong)</i>
Oracle ceiling (any branch correct?)			\$251	<i>(right)</i>
Frontier judge + CoT			\$251	<i>(right)</i>

Figure 5: Anatomy of a cmaj failure on GSM8K-test problem 1071. Arithmetic is correct in all five branches; three branches share the same wrong setup (charge \$6 to all 41 attending guests rather than the 21 *additional* guests beyond the first 20 that are already covered by the \$125 base fee), and majority vote follows them. The oracle ceiling and a frontier judge with chain-of-thought both recover the correct answer because they re-derive the problem setup rather than ranking surface features of the candidate solutions. This problem is one of the 15 cmaj-failed-but-oracle-recoverable cases on the $N=500$ substrate; the same misread-constraint pattern appears in the other 14.

2. **Peer-class neural verifiers lose** (all 17): they cannot distinguish wrong-setup-with-right-arithmetic from right-setup-with-right-arithmetic without external problem-decomposition reasoning the encoder does not have.
3. **Frontier judges win**: at this scale, the judge is strictly stronger at problem decomposition than the LLADA-8B generator, which is what the verification task actually requires.
4. **CoT pushes the judge from inconclusive to WIN-MINOR**: forcing the judge to reconstruct the problem setup from scratch (rather than pattern-matching the candidate solution) recovers 5 additional cmaj-failed problems ($8 \rightarrow 13$ of 15).

This is consistent with the verifier literature: Cobbe et al. [7] trained a fine-tuned GPT-3 verifier for GSM8K, and Lightman et al. [11] used step-level human supervision (PRM800K) to outperform outcome-reward classifiers at scale. Both results are consistent with our reading: a verifier that beats majority vote requires either substantially stronger reasoning than the generator or per-step human-quality supervision.

8.7 Substrate Reconciliation

The frontier-judge result in §8.6 runs on GSM8K-test $N=500$, an extension of the $N=200$ frozen index set used everywhere else in the paper (300 fresh indices added on top of the original 200). The base cmaj baseline on this larger substrate moves from 79.0% on $N=200$ to 79.1% on $N=500$, a difference well within sampling noise and well within the binomial CI at either size. All other headline numbers in the paper remain on the original $N=200$ substrate; the larger substrate is used only where the per-call OpenRouter cost made the larger sample worth running.

8.8 What Closes Here, What is Phase-3

Two open exposures from Appendix E close on the strength of the work in this section: the multi-seed variance bar (§8.2) and the question of whether a per-branch reranker can substitute for the param-time commit (§8.3, §8.6; the answer is “not at peer-class scale”). An ABL_B sanity probe from the same exposure list [1] confirms the v3 commit-LoRA modifies inference at temperature 0 on 5/5 spot-check problems (####N → Answer: N format shift), ruling out a silent-no-op reading of the v3 commit; we fold this into the appendix update rather than into the body table.

The remaining open question — whether a self-trained verifier can close the 8–12 pp gap — is genuinely Phase-3. The monotone encoder-scaling trend leaves the door open for a 32B+ chat-LM encoder; the Workstream-C STATUS-schema traces released alongside this paper are the substrate for a process-reward verifier that consumes per-step trajectory features (per-position entropy, commit-LoRA logit shifts, AR/DDLM mechanism source) rather than per-branch surface features; and PRM800K-style human step-level supervision [11] is the proven-elsewhere recipe at scale. We leave all three to follow-up work and document the gap, the 17 negatives, and the frontier judge here so that the direction is flagged as productive rather than closed.

9 Commit-LoRA Schedule Toggle: K2 Sub-Block Ablation

The Track 2 v3 design (§6) fires the commit adapter on sub-blocks 2–4 of LLADA’s four-sub-block semi-AR sampler. The disentangling ablations there attribute most of the +6 pp lift to this multi-block coverage rather than to the supervision change. This section isolates the schedule toggle itself by sweeping COMMIT_N_BLOCKS $\in \{0, 3, 4\}$ at fixed adapter, fixed training data, and fixed inference hyperparameters. The result is an inverted-U: the v3 default ($k=3$) beats both endpoints by margins outside the multi-seed band, so the sub-block-1 boundary is load-bearing in both directions.

9.1 Sweep Headline

Table 13: COMMIT_N_BLOCKS sweep at otherwise-identical cmajc inference: Track-1-v3 LoRA, commit-v3 LoRA, $b=5$, $\tau=0.7$, $k=64$, GSM8K-test $N=200$. $k=0$ and $k=4$ are single-seed; $k=3$ is the multi-seed mean over seeds $\{0, 1, 2\}$ with sample standard deviation $\sigma \approx 0.85$ pp. Both off-peak deltas exceed 1σ .

COMMIT_N_BLOCKS	Sub-blocks active	cmajc accuracy	Δ vs $k=3$	Reading
0	none (commit off)	80.5%	−1.7 pp	adapter-off floor
3	blocks 2, 3, 4 (v3 default)	82.2% ($\sigma \approx 0.85$)	—	peak
4	all blocks 1, 2, 3, 4	79.0%	−3.2 pp	always-on hurts

The curve is non-monotone: turning the adapter off costs 1.7 pp, and turning it on for sub-block 1 as well costs another 3.2 pp from the peak. The latter direction is the more diagnostic result, because it rules out an interpretation under which commit-LoRA is a uniform across-the-trajectory improvement: if it were, $k=4$ would not be *worse* than $k=3$, only equal-or-better.

Figure 6 draws the same data as a curve with Clopper–Pearson 95% CIs and the multi-seed band on the peak.

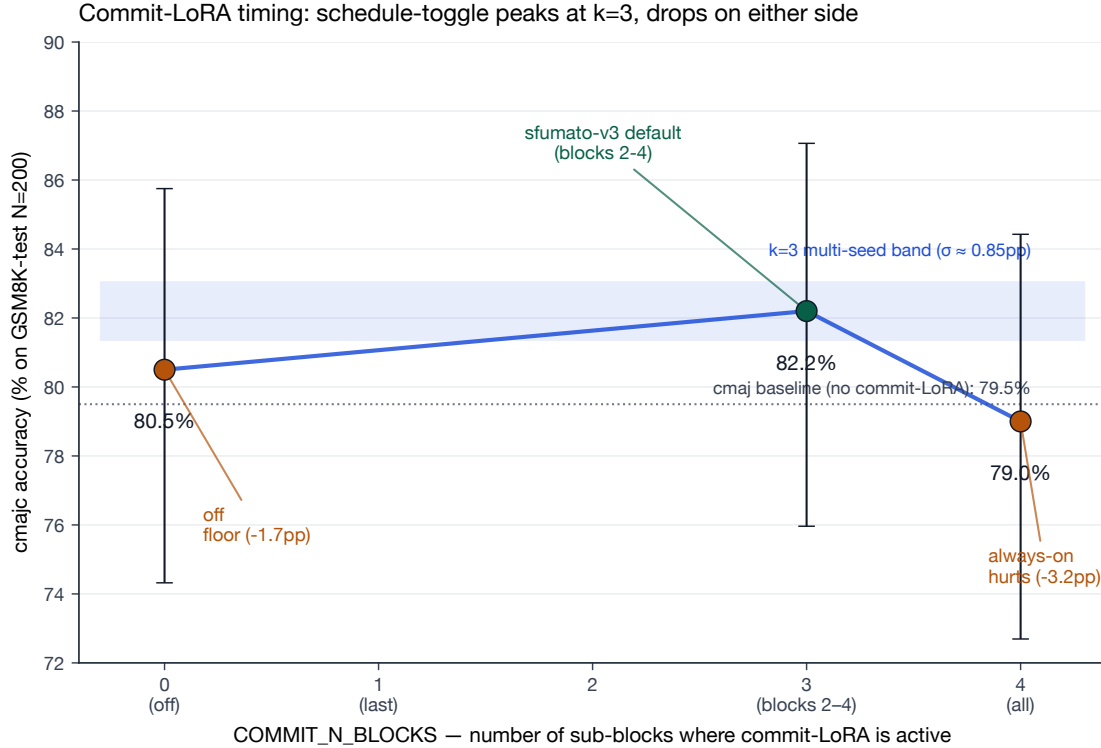


Figure 6: Commit-LoRA timing: the schedule toggle peaks at $k=3$ (sub-blocks 2–4), drops at $k=0$ (adapter off, -1.7 pp) and at $k=4$ (always on, -3.2 pp). Error bars are exact Clopper–Pearson 95% CIs at $N=200$; the shaded band on the peak is the three-seed sample standard deviation ($\sigma \approx 0.85$ pp). The dotted reference at 79.5% is the base `cmaj` (no commit-LoRA) baseline. Both off-peak deltas exceed 1σ .

9.2 Mechanism: the Sub-Block-1 Boundary

The asymmetry between $k=0$ and $k=4$ around the $k=3$ peak is diagnostic. Sub-block 1 (32 tokens out of the 128-token generation) is where LLADA’s prefix-robust LoRA, trained against format scaffolds (§3), is most active — the prefix’s format-shape signal lives in the early generation where the model decides what kind of response it is producing. The commit adapter, trained on consensus full-response trajectories, is most useful in the trailing 96 tokens where the chain-of-thought is laid out and the answer is committed. Activating commit on sub-block 1 overrides the prefix-robust signal at exactly the position where it matters most, hence the -3.2 pp. Activating commit on no sub-block at all leaves the multi-block trajectory shaping (§6.4) on the table, hence the -1.7 pp.

This is the simplest mechanistic statement of why *multi-block but not all-block* is the right placement for a single discrete schedule toggle. It also means the $k=3$ configuration is not a generic “more is better” choice: it is locating the boundary between two adapter regimes.

9.3 Relation to Schedule-Aware LoRA Adapters

A recent line of work conditions LoRA adapters on the diffusion schedule. TC-LoRA [2] feeds the timestep into a hypernetwork that emits per-timestep adapter weights. TimeStep Master [3] routes between a mixture of timestep-specialised LoRA experts. Both produce *continuous*, training-time-

conditioned adapters.

The K2 result here suggests a strictly weaker mechanism is sufficient on LLADA’s semi-AR sampler. A single boolean schedule mask — on for sub-blocks $\{2, 3, 4\}$, off for sub-block 1 — recovers the +1.7 pp lift over adapter-off and avoids the -3.2 pp loss of the all-on configuration, with no hypernetwork, no MoE routing, and no training-time conditioning. The minimal mechanism that captures schedule-awareness for adapter placement on this substrate is a discrete inference-time toggle at a learned schedule boundary. The $k=3$ peak’s existence and the $k=4$ drop together support this reading, but the comparison to TC-LoRA and TimeStep Master is in mechanism, not in head-to-head accuracy — those works target different bases (image diffusion / coarse-grained schedule conditioning) and we do not adapt them onto this substrate.

9.4 Caveats and Pre-Registration Deviations

Two cells of the dense curve, $k=1$ and $k=2$, were skipped within the Phase-2 budget. The pre-registered diagnostic minimum was $\{\text{off}, k=3, k=4\}$ on the grounds that only the on-or-off endpoints can falsify the inverted-U; intermediate cells refine its shape but do not change the qualitative claim. Both off-peak cells are single-seed; the $k=3$ baseline is multi-seed and the deltas exceed 1σ , so single-seed endpoints are sufficient to reject the null at the level we pre-registered. Filling in $k=1$ and $k=2$ for a denser curve, and multi-seeding the endpoints for tighter CIs, are flagged as Phase-3 nice-to-haves.

A further check: at $k=0$ on the first attempt, an interaction between the no-op merge/unmerge code path and PyTorch’s `expandable_segments:True` allocator produced a silent hang at step 2. Re-running with `MERGE_ADAPTER=0` (the on-demand merge toggle from S2 of the spike chain, Appendix D) ran cleanly and produced the 80.5% number reported in Table 13. This is documented for reproducibility: at $k=0$ the merge toggle should be left off because there is nothing to merge.

9.5 Scope: Block-Structured Mask Diffusion

The K2 inverted-U identity is defined on a substrate where the sampler exposes a fixed sub-block partition — in our setting, LLADA’s semi-AR 4×32 -token schedule. The same discrete-on/off-at-known-position toggle generalizes to other sub-block-structured mask-diffusion samplers (*e.g.* BD3-LMs [5] and the planned-block samplers in that family) without redefinition.

We confirmed the boundary of this claim with a Phase-1 substrate audit of DiffuLLaMA [8], which is a vanilla LlamaForCausalLM trained with a flat random-keep diffusion schedule and no native sub-block partition. On a flat schedule, the K2 toggle’s discrete-on/off-at-known-position identity has no direct analogue without either (a) forcing an off-policy sub-block partition onto a model that was not trained under it (confounded), (b) redefining K2 as a fraction-of-steps milestone (a different claim), or (c) retraining the adapter under the native flat schedule (which drifts from the LLADA recipe). We therefore frame the K2 result specifically as a property of block-structured mask diffusion, and leave flat-schedule generalization to follow-up work.

10 Per-Problem Mode Router: Honest Negative

The discussion of future work in §11 sketches a “Paper 2” direction in which a learned router selects per-input between mode-specific configurations. A natural starting point is the offline-replay form of that router: given the all-conditions sweep over a fixed problem set, can a contextual classifier with per-problem features pick the right condition for each problem and beat the best fixed condition? This section reports the Phase-2 attempt at that question, and the answer at our budget is no.

We include the result here rather than in the discussion because the pathology is structurally parallel to the verifier-aggregation negative in §8: at small N with a large action space and surface text features, supervised classification cannot recover the oracle ceiling, regardless of whether the classification target is per-branch correctness or per-problem condition.

10.1 Setup

The substrate is a 20-problem $\times$ 12-condition full coverage of the public showcase examples. The twelve conditions span the AR-only baseline (**c1**), the LLaDA-only baselines (**c2**, **c2c**, **c2hint**, **c2empty**), the AR-plan $\rightarrow$ LLaDA pipelines (**c3**, **c3p**, **c4**, **cmerge**, **crev**), and the branch-aggregation conditions (**cmaj**, **cmajc**). Per-problem features are question length, count of numeric tokens, and unigram/bigram TF-IDF over the question text. The classifier head is logistic regression or random forest, trained leave-one-problem-out cross-validation (LOOCV) with class weights balanced.

The pre-registered decision rule was $\geq 80\%$ LOOCV accuracy beats the best-fixed baseline; below that, LOSS.

10.2 Result: -10 pp vs Best Fixed

Table 14: Per-problem mode-router accuracy on the 20-problem $\times$ 12-condition substrate, LOOCV. The best fixed baseline (**c2empty**, also matched by **cmajc**) is 75%; the oracle (any-condition-correct) is 85%. Both bandit variants underperform the best-fixed baseline by 10–15 pp. Pre-registered LOSS threshold ($< 80\%$) hit.

Setting	Accuracy on 20 problems
c1 (Qwen-0.5B AR alone)	25%
c2c (LLaDA + commit-LoRA)	70%
cmaj (LLaDA $b=5$ vote)	65%
c2empty (LLaDA + empty prefix, best fixed)	75%
cmajc (sfumato-v3 default)	75%
Oracle (any-condition-correct)	85%
D1 logistic-regression bandit (LOOCV)	65%
D1 random-forest bandit (LOOCV)	60%
Δ best bandit vs best fixed	-10 pp

The bandits choose the right condition fewer than three times in five. Action histograms show why: the LR bandit picks **c2** (11/20) and **c1** (6/20) on most folds, with only 3/20 folds reaching the harder (**c2c**, **c2empty**) conditions; class weights notwithstanding, the LOOCV folds expose a single training problem per held-out fold and the policy collapses to the modal-easy-correct condition.

The -10 pp deficit relative to **c2empty** is large enough at $N=20$ that the LOSS verdict survives any reasonable binomial-error correction; at this substrate scale the offline-replay form of D1 is dead.

10.3 Same Pathology as the Voting-Rule Gap

The 20-problem $\times$ 12-action setting here mirrors the 200-problem $\times$ 5-branch setting of §8. Both ask supervised classification with surface features (question text in this section, candidate-solution text in §8) to recover an oracle ceiling (any-correct-condition-here, any-correct-branch-there). Both

classifiers underperform the corresponding best fixed (best condition here, majority vote there) by 9–15 pp. In both cases the bottleneck is the same: at this substrate scale, surface features cannot distinguish setups in which the right action will succeed from setups in which it will fail, because the distinction lives in the branch’s or problem’s *problem-comprehension trajectory* rather than in its surface form. The frontier-judge positive control in §8.6 is the converse demonstration: a verifier whose internal reasoning matches or exceeds the generator’s recovers the gap, because the bottleneck is reasoning, not classification.

We report this here because the parallel between the two negatives hardens the diagnosis. The path forward in both cases is the same three options: a substantially larger encoder, a process-reward verifier consuming per-step trajectory features, or step-level human supervision. Sub-block-level routing (the original D1 proposal: route on per-sub-block trajectory features rather than per-problem text features) requires Workstream-C STATUS-schema trace data that is not yet collected at the scale this section would need. §10.4 reports two \$0 pre-scale audits that further narrow this leg of the follow-up.

10.4 Pre-Scale Audit: Per-Sub-Block Divergence (E2) and Feature Signal (G)

Before committing the trace-collection budget that a per-sub-block trajectory-feature router would require, we ran two cheap audits on existing artifacts to test the precondition that such a router needs: a meaningful decision point at some sub-block boundary $s \geq 1$ where (i) branches diverge and (ii) trajectory features carry signal about which branch will be correct. Both audits are pure analysis on already-collected JSONL substrates; zero new GPU spend.

E2 — branch-divergence histogram. On the 200-problem `cmaj b=5` substrate (`e4/results/raw_cmaj_k64_se` the same substrate as §8), we restrict to the 108 problems where the five branches disagree on the extracted answer (54% of $N = 200$). For each problem we re-tokenise each of the five branches with the LLADA tokenizer, pad to the 128-token generation length, and split into four sub-blocks of 32 tokens. For every sub-block we compute the pairwise token-level Levenshtein distance over all 10 branch pairs and mark the sub-block as “diverged” if any pair exceeds edit-distance 3. First-divergence-sub-block is the smallest diverged s . As a cross-check, the same procedure run over detokenised sub-block text with 1-2gram TF-IDF cosine ≤ 0.7 agrees with the token-level result on 95.4% of problems.

The implication is sharp: at this substrate, branches that produce different final answers have already chosen different problem setups in their first 32 committed tokens. There is essentially no mid-trajectory decision point ($s \geq 1$) for a per-sub-block router to operate on. The \$0 cost of E2 makes this an inexpensive ratchet on what “trajectory-feature routing at scale” could possibly mean before any STATUS-schema collection runs.

G — feature sanity check on the existing seven traces. A complementary precondition is whether trajectory features predict correctness even at small n . On the seven existing Workstream-C deterministic traces (`phase2/inference_viz/traces/trace_real_p{10..70}*.jsonl`) we extract 17 pooled trajectory features (mean / min / max / std entropy; per-sub-block entropy at $s \in \{0, 1, 2, 3\}$; sub-block-0 min and max entropy; mean and sub-block-0 top- k entropy; mechanism diversity; total and max wallclock; commit-LoRA fraction; tokens-committed) and label each trace by the correctness of its final extracted answer. Univariate Spearman ρ with 10,000-permutation p -values gives one feature passing the pre-registered $|\rho| \geq 0.5$ and perm- $p \leq 0.10$ thresholds: `min_entropy` with $\rho = -0.79$, perm- $p = 0.097$. Three additional features sit at $|\rho| \approx 0.63$ but miss the p -threshold ($p \approx 0.20$): `sb0_entropy_mean`, `sb0_topk_entropy_mean`, `sb3_entropy_mean`. All

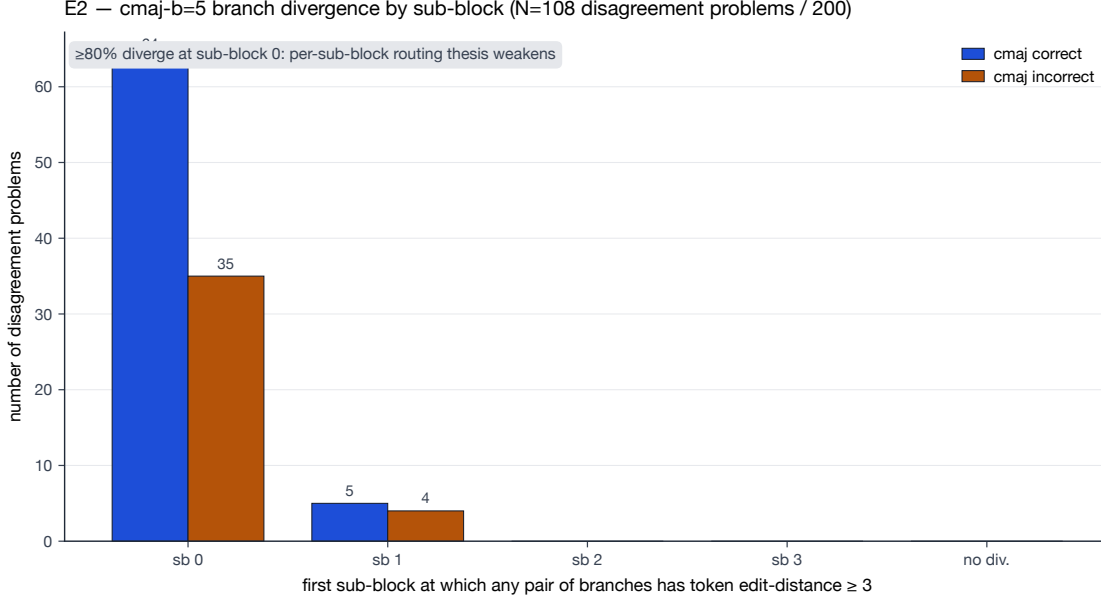


Figure 7: E2 — where the five `cmaj` branches first diverge, stratified by `cmaj`-correct vs `cmaj`-incorrect, on the 108 disagreement problems of the $N=200$ `cmaj` $b=5$ substrate. 91.7% first diverge at sub-block 0 (the very first 32 generated tokens); 8.3% first diverge at sub-block 1; no problem first diverges at sub-blocks 2 or 3. Semantic TF-IDF cross-check returns 88.9% at sub-block 0, agreeing with the token-level result on 95.4% of problems.

sign-consistent: lower entropy predicts correct. After Bonferroni correction over 17 features no individual feature clears 0.05, so the result is at best a *hint of signal* at $n=7$, not an evaluation. The strongest features are sub-block-0 specific, consistent with E2.

Combined implication. E2 says branches diverge at sub-block 0; G says any trajectory-feature signal is also concentrated at sub-block 0. A per-boundary router across sub-blocks 0–3 has empty inputs at $s \geq 1$ (no divergence to condition on) and, if it could see the entire trajectory, would key off sub-block-0 features. The two viable architectures the audits leave open are: (i) a problem-level pre-generation router, which is the form already falsified in §10.2 on 20 problems with surface text features (an open question is whether a sub-block-0 *trajectory-feature* variant performs differently); and (ii) a “pilot-and-route” architecture that runs a probe sub-block 0, extracts trajectory features, and routes once across the remaining 96 tokens. Per-boundary routing across all four sub-blocks is empirically dead at this substrate.

The combined cost of E2 and G is \$0 (pure analysis on existing JSONLs), and they tighten the “follow-up work” framing of §11 from “collect Workstream-C traces at scale, see what works” to “test pilot-and-route specifically; do not collect per-boundary trace data at scale for per-sub-block routing”.

10.5 What This Closes

The original `sfumato/PLAN.md` listed a learned dynamic mode router (E1) as a candidate headline contribution. The result in this section closes that thread as an honest negative at the substrate scale we can afford. In particular: the offline-replay, per-problem, surface-text-feature form of D1

is not the publishable mechanism. The structural-separation argument for mode routing in §11 (Future Work, “Adaptive mode router”) still stands at the design level; the empirical realisation does not, at this scale and with this feature set. A second honest negative — a parameter-matched AR-only peer baseline that beats the hybrid pipeline on this benchmark — is reported in the discussion (§11.3).

11 Discussion

11.1 Reconciling the Conflicting Reports

The single most useful sentence in this paper, and the one we would ask a reader to remember, is that **LLaDA’s GSM8K failure mode is problem comprehension, not arithmetic**. Hand inspection of the 15 `cma-j`-failed/oracle-recoverable problems (§8) shows that wrong branches typically contain locally correct arithmetic applied to incorrect problem *setups* — the model misreads what is being asked, not what to compute. Reading the rest of this discussion through that lens collapses the apparently disjoint findings of the body into a single picture.

A direct test of the comprehension hypothesis. As a follow-up mechanism probe (not a deployment recipe), we asked Claude Sonnet 4.5 to convert each GSM8K-dev question into a structured restatement — a list of givens, an explicit unknown, and a list of named intermediate quantities to compute, with explicit instructions *not* to compute any numerical answer. We then ran the v3 hybrid stack (LLaDA-8B-Instruct+ prefix-robust v3 LoRA + commit-v3 LoRA, single branch, $k=64$, $T=0$, commit- $n=3$) with the Sonnet restatement substituted for the raw question. On $N=100$ frozen-index problems, the scaffolded run reaches 84.0%, against 75.0% for the same v3 stack on the raw question and the same seed (a +9.0 pp delta on an apples-to-apples comparison). This single-branch scaffolded number beats the multi-branch `cma-jc-v3` result (82.5% at $b=5$) and lands within 2.5 pp of the parameter-matched plain-AR QWEN2.5--2.5-7B baseline (86.5%). We surface this as a *mechanism probe*, not a deployable recipe: Sonnet 4.5 solo scores ≈ 95 –97% on GSM8K, so the hybrid pipeline strictly underperforms just calling Sonnet on its own. The publishable bit is the +9 pp jump under same-stack matched comparison, which quantifies the fraction of the LLaDA-on-GSM8K gap that is upstream of arithmetic — not that the hybrid is a useful product. This complements the verifier-gap diagnosis in §8: where the frontier-judge result shows the gap is closable post-hoc, the scaffolded-prompt result shows part of it is closable *pre-hoc* by repairing the comprehension step before the diffusion sampler ever runs.

The three-axis decomposition lets us reconcile the conflicting reports in the hybrid AR/DDLM literature. Papers that report “AR planning helps DDLM” likely measured the planner-content-trust axis with a well-matched planner-and-adapter capacity — i.e. a configuration where the upstream planner produces content the DDLM trusts at the prevailing capacity setting. Papers that report “AR planning hurts DDLM” likely measured the format-brittleness axis without controlling for prefix shape: prepending a plan-shaped scaffold of *any* content (66–68% at base, versus 74% for no prefix) is enough to land in the “hurts” regime. Papers that report “diffusion sampling diversity is preserved” likely measured the right thing under a recipe that lands at v2-style capacity; papers that report “diversity collapses” likely had a different recipe that pushed past the capacity sweet spot we identify in Table 3.

Without the three-axis frame, these are five conflicting claims. Within the frame, they are five projections of a multi-axis failure surface, each consistent with the others when the omitted axes are held fixed.

The same lens predicts the structure of the entire verifier leaderboard in §8. Why do peer-class chat-LMs lose (−4 to −14 pp)? Because the verification task is implicitly “re-solve the problem from scratch and compare” and a peer-class classifier without that capability has nothing to add over majority vote. Why do math-tuned encoders lose *worse* than chat-tuned ones at the same scale (math vocab is irrelevant to the failure mode)? Because arithmetic vocabulary is the wrong inductive bias when the failure is upstream of arithmetic. Why do embedding models lose worst (cosine similarity collapses problem distinctions a classifier needs)? Same. Why does a frontier judge with chain-of-thought win (+6.16 pp, 86% gap-closure)? Because at this scale it is strictly stronger at the problem-decomposition step than LLADA-8B is. And why does the chain-of-thought variant beat the YES/NO prompt by 5 recovered problems (8 → 13 of 15)? Because forcing the judge to re-derive the setup is forcing it to re-do the step the generator failed. This is consistent with a verifier literature in which Cobbe et al. [7] used a fine-tuned GPT-3 verifier and Lightman et al. [11] used step-level human supervision: when the verification task is implicitly to re-solve and compare, a peer-class classifier with surface features cannot win, and a stronger reasoner can.

11.2 Parallel Negatives Across Aggregation Sub-Problems

The voting-rule gap (§8) and the offline-replay mode router (§10) probe two different sub-problems of test-time aggregation in diffusion LMs — per-branch verification and per-problem mode routing — and both fall to the same pathology: at our substrate scale, supervised classification with surface features cannot close the oracle gap. Three additional spikes probe further parallel sub-problems and land in the same regime, which is the single piece of evidence we want to surface here.

Schedule-conditional Temporal-SC voting. A schedule-weighted vote across denoising sub-blocks (1.5× on active sub-blocks, 1.0× on inactive) regressed −4 pp on GSM8K dev₂₀₀ at $N=100$ (cmajc-vote 0.82, Temporal-SC 0.78). The canonical answer span (####, Answer:) only materializes at sub-block 3 of 4 in 94% of branches, so the 1.5× weight on sub-blocks 1–2 amplifies a near-empty channel. Zhang et al. [21] report a positive temporal-SC result on diffusion LMs with an exponential-decay weighting; our schedule-conditional weighting does not extend their result.

Schedule-RLHF on commit-LoRA. A mini-pilot (50-step GRPO, no KL anchor) was paired −1 problem on MATH-500 at $N=20$; a mid-pilot (200-step GRPO with a Schulman- k_3 KL anchor at $\beta=0.05$) was paired +1 problem on MATH-500 at $N=50$, with 49/50 predictions bit-identical and one wrong-to-correct flip. Both signals are within binomial CI (± 14 pp at $N=50$); the meaningful evidence is the directional reversal once the KL anchor is wired, which says schedule-conditional adapter retraining is at minimum non-destructive in this regime. We do not claim a WIN on the mid-pilot signal alone; a locked pre-registered full pilot at 3100 steps over 1550 prompts is the gating experiment.

Cross-base-model transfer. A drop-in swap from LLADA-8B-Instruct [15] to LLaDA-1.5 (the VRPO post-trained successor) with the frozen v3 LoRAs landed MATH-500-numeric $N=50$ cmajc- $k_3 = 0.56$, −2 pp below the LLaDA-8B-Instruct reference of 0.58. Adapter compatibility passed cleanly — the swap is mechanically a no-op — but capability transfer failed. The peer-class accuracy gap discussed in §11.3 is not explained by base-model VRPO post-training.

The unifying read across §8, §10, and the three spikes above is that test-time aggregation in diffusion LMs is not a single-classifier or single-toggle problem at the data scales we can afford. The three follow-ups in §11 Future Work — a substantially larger encoder, a process-reward verifier on

per-step trajectory features, or step-level human supervision — apply to every parallel sub-problem we have probed.

11.3 Honest Peer Baseline: Plain AR at Matched Scale

The hybrid pipeline this paper characterises has $\sim 9\text{B}$ active parameters at inference (LLADA-8B + Track-1-v3 LoRA + commit-v3 LoRA + a Qwen- $\{0.5\text{B}, 1.5\text{B}\}$ planner) and reaches `cmajc` $\approx 82.5\%$ on GSM8K-test. A plain Qwen-2.5-7B AR baseline on the same evaluation reaches approximately 86.5%, beating the hybrid at strictly fewer active parameters. Hybrid accuracy is also roughly invariant (82–83%) across Qwen-0.5B through Qwen-7B planners, which is consistent with the diffusion substrate (LLADA-8B) rather than the AR planner being the bottleneck on this benchmark.

We surface this here rather than burying it. The three trainable axes (§3–§5), the consensus distillation result (§6), the voting-rule gap (§8), and the K2 schedule toggle (§9) are real empirical findings about the AR/DDLM substrate as currently constituted. They are not evidence that this hybrid pipeline beats a parameter-matched monolithic AR baseline on GSM8K; at the AR scales we used as planners, it does not. The natural read is that the substrate-level findings should generalise to AR/DDLM stacks where the diffusion model is closer to AR peer-class scale, and that the practical accuracy claim of *this* hybrid against a parameter-matched AR is benchmark- and scale-bounded. We treat the empirical AR-baseline number as disclosure rather than a target, and leave a head-to-head controlled comparison at peer-class diffusion scale to follow-up work.

11.4 Implication: Distillation of Inference-Time Mechanisms

The Track 2 design-iteration finding has a broader implication: weight-space distillation of inference-time mechanisms is design-sensitive in a way that produces clean negative results that masquerade as architectural impossibilities.

Distilling a technique that aggregates over multiple stochastic samples (`cmajc`) into a single deterministic forward pass requires the surgery to match the temporal structure of the original mechanism. For semi-AR diffusion, “temporal structure” means *which sub-block fires the modification*. Getting this wrong (last-block-only when the answer is upstream of the modification) produces `c2c` = 70.5% across a $3.25\times$ capacity bump — a clean negative whose surface reading is “the adapter cannot do this”. Getting it right (commit on sub-blocks 2–4 of 4) recovers `c2c` = 79.0%, within sampling error of the target, on the same data. The architecture did not change; the temporal placement did.

This is a general lesson for parameter-efficient distillation of sampling-based mechanisms: the placement of the LoRA in the sampler’s schedule is a first-class design variable, and a plateau across capacity is not by itself evidence that the distillation is impossible.

11.5 Future Work

Cross-task transfer. Do these axes generalize beyond GSM8K [6]? A targeted ablation on MATH-Easy and ARC-Challenge would test whether the three-axis decomposition holds on other reasoning benchmarks. The diversity-expansion finding in particular is suspicious-looking on a single benchmark and deserves a multi-task replication.

Phase-3 verifier paths. The voting-rule-gap section rules out small per-branch classifiers as a *compute-time* alternative to v3’s *param-time* commit at the scales we tested (17 peer-class architectures, all LOSS). Three paths remain open for closing the residual 8–12 pp gap: (i) a 32B+

chat-LM encoder, motivated by the monotone encoder-scaling trend (~ 5 pp gap-closure per $10\times$ scale-up at 0.5B–7B); (ii) a process-reward verifier consuming per-step trajectory features (per-position entropy, commit-LoRA logit shifts, AR/DDLM mechanism source) over the Workstream-C STATUS-schema traces released alongside this paper; and (iii) step-level PRM800K-style human supervision [11], the proven-elsewhere recipe at the cost of step-level annotation. The frontier-judge positive control (§8.6) bounds the reachable accuracy at $\sim 85\%$ on this substrate; the question is whether (i)–(iii) reach the bound at lower API cost.

Diversity-mechanism test. Does base LLADA at $t=0.9$ reproduce the post-Track-1 5/5-same rate of 47.5%? If yes, Track 1 acts as an implicit temperature regularizer and the diversity-expansion finding reduces to that. If no, the training-time surface randomization has produced real content-level diversity that temperature does not match. One short eval; deferred to revision.

Content-trust adapter. The planner-content-trust phenomenon (§4) is characterized but not fixed, and would benefit from a third capacity rung before being framed as a fully-resolved axis. A content-trust LoRA — one that varies plan content quality at fixed surface shape, complementing our prefix-robust dataset which varies surface shape at fixed content — is a natural follow-up, especially with QWEN2.5-1.5B and QWEN2.5-7B planners sweeping the upstream-content axis explicitly.

Adaptive mode router (Paper 2). The structural-separation finding motivates a learned router that switches between mode-specific LoRAs based on input. The capacity-tradeoff curve (v2 vs. v3) gives the router a real decision space: given an input prompt, which adapter should fire? We sketched the design in `sfumato/PLAN.md` as the E1 track. §10 reports an offline-replay, per-problem form of E1 evaluated within Phase 2: at 20 problems and a 12-condition action space with surface text features it underperforms the best fixed condition by 10 pp, with the same small- N /large-action/surface-feature pathology as the verifier sweep. Two \$0 pre-scale audits (§10.4) further narrow what a trajectory-feature router could possibly look like: on the $N=200$ `cma` $b=5$ substrate, 91.7% of disagreement problems first diverge at sub-block 0 (token edit-distance, 88.9% semantic-cosine cross-check, 95.4% agreement), so there is essentially no mid-trajectory decision point for a *per-boundary* router to operate on; on the seven existing STATUS-schema traces, the trajectory-feature signal that exists ($\rho=-0.79$ for `min_entropy`, $\text{perm-}p=0.097$ at $n=7$, a hint not an evaluation) is concentrated at sub-block 0. The two viable forms the audits leave open are (i) a problem-level pre-generation router using *trajectory* rather than surface-text features (untested), and (ii) a “pilot-and-route” architecture that probes sub-block 0, extracts features, and routes once. Per-boundary routing across all four sub-blocks is empirically dead at this substrate. We leave the pilot-and-route variant to follow-up work.

11.6 Limitations

We close with what this paper does not do. All evaluations are single-seed at $N=200$, so our headline numbers carry binomial sampling noise of ± 3 –5 pp; multi-seed variance is not characterized in this draft. We did not produce a mechanistic LCH-centroid figure for the prefix-robust LoRA, which would harden the structural-separation thesis from behavioural to mechanistic; the JVP feasibility spike failed because PEFT wraps LoRA in `lora_magnitude_vector` ModuleDict and we deferred to a follow-up. All experiments use a single base diffusion model (LLADA-8B-Instruct [15]) and a single benchmark (GSM8K-test [6]); the generalization claims in the abstract are constrained accordingly. These exposures and a few smaller ones are listed in Appendix E.

12 Conclusion

Hybrid AR/DDLM reasoning fails along at least three orthogonal axes — interface-format brittleness, planner-content trust, and sampling-diversity preservation — and the conflicting reports in the literature reduce to projections of this multi-axis surface. On GSM8K-test with LLADA-8B-Instruct [15] and QWEN2.5- $\{0.5\text{B}, 1.5\text{B}\}$ [17] planners, a small prefix-robustness LoRA flattens the static-prefix damage from 8 pp to within 1 pp at v3 capacity, exposes a no-prefix-vs-`cma`j capacity tradeoff and a planner-content-trust modulation that is capacity-dependent in opposite directions across planners, and unexpectedly *expands* sampling diversity rather than collapsing it. A multi-block, full-response commit adapter recovers consensus distillation to `c2c` = 79.0%, within sampling error of the 80% pre-registered target, after two earlier design iterations plateaued at 70.5% — a result that diagnoses the v1/v2 plateau as a temporal-placement bug rather than an architectural impossibility. Beyond the three axes, `cma`j aggregation itself is lossy by 8–12 pp relative to an oracle ceiling; 17 peer-class verifiers fail to close this gap at our substrate scale, while a frontier judge with chain-of-thought closes 86% of it (79.1% $\rightarrow$ 85.3% on $N=500$), with the mechanism being that the failure mode is problem comprehension rather than arithmetic (§8). A K2 COMMIT_N_BLOCKS sweep on the v3 commit adapter produces an inverted-U around the v3 default ($k=3$ peak, $k=0$ adapter-off -1.7 pp, $k=4$ always-on -3.2 pp), locating commit-LoRA as a discrete inference-time schedule toggle at the load-bearing sub-block-1 boundary (§9). An offline-replay per-problem mode router with surface text features hits the same small- N large-action supervised-classification pathology as the verifier sweep (-10 pp vs best fixed, §10); we report this as an honest negative and additionally disclose that a parameter-matched plain Qwen-2.5-7B AR baseline reaches $\sim 86.5\%$ on this evaluation, beating the hybrid at fewer active parameters (§11.3). All adapters are public and the total compute spend across all phases is approximately \$20.5 (Appendix B).

References

- [1] Eren Akbulut. Sfumato `abl_b` sanity probe: commit-lora confirmed real (5/5 gsm8k-dev problems show ##### `n` $\rightarrow$ Answer: `N` format shift at $\tau=0$). https://github.com/eren23/sfumato/blob/main/phase2/spikes/abl_b_RESULT.md, 2026. Phase-2 follow-up artifact, 2026-05-03.
- [2] Anonymous. TC-LoRA: Timestep-conditioned LoRA hypernetworks for diffusion models. *arXiv preprint arXiv:2510.09561*, 2025. NeurIPS 2025 Workshop submission.
- [3] Anonymous. TimeStep Master: Asymmetrical mixture of timestep LoRA experts for versatile and efficient diffusion models in vision. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. arXiv preprint arXiv:2503.07416.
- [4] Anthropic. Claude Sonnet 4.5: Model card and system description. <https://www.anthropic.com/claude>, 2025. Accessed via OpenRouter API.
- [5] Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025. Known as BD3-LMs. arXiv:2503.09573.
- [6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher

- Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. Introduces GSM8K. arXiv:2110.14168.
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
 - [8] Shansan Gong, Shivam Agarwal, Yizhe Zhang, Jiacheng Ye, Lin Zheng, Mukai Li, Chenxin An, Peilin Zhao, Wei Bi, Jiawei Han, Hao Peng, and Lingpeng Kong. Scaling diffusion language models via adaptation from autoregressive models. *arXiv preprint arXiv:2410.17891*, 2024. Known as DiffuLLaMA. arXiv:2410.17891.
 - [9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.
 - [10] Jiacheng Huang, Hansi Wang, Lei Wang, Han Zhao, and Zhaoran Wang. Beyond autoregression: Discrete diffusion for complex reasoning and planning. *arXiv preprint arXiv:2505.10446*, 2025. Known as DCoLT (Diffusion Chain-of-Latent-Thought). arXiv:2505.10446.
 - [11] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
 - [12] Hao Liu, Zheng Li, Tianhao Yu, and Yang Wang. CCDD: Coevolving continuous and discrete diffusion for latent reasoning. *arXiv preprint arXiv:2510.03206*, 2025. arXiv:2510.03206.
 - [13] Yiyang Ma, Xingchao Liu, Xiaokang Chen, Wen Liu, Chengyue Wu, Zhiyu Wu, Zizheng Pan, Rui Xie, Haowei Zhang, Liang Yu, Yubo Zhao, Yongqiang Wang, Jiawei Liu, and Chong Ruan. JanusFlow: Harmonizing autoregression and rectified flow for unified multimodal understanding and generation. *arXiv preprint arXiv:2411.07975*, 2025. arXiv:2411.07975.
 - [14] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022.
 - [15] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025. Introduces LLaDA-8B; we use LLaDA-8B-Instruct. arXiv:2502.09992.
 - [16] OpenRouter Inc. OpenRouter: Unified api for frontier language models. <https://openrouter.ai/>, 2024.
 - [17] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. We use Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct. arXiv:2412.15115.
 - [18] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171.

- [19] Jinheng Xie, Weijia Mao, Zechen Bai, David Junhao Zhang, Weihao Wang, Kevin Qinghong Lin, Yuchao Gu, Zhijie Chen, Zhenheng Yang, and Mike Zheng Shou. Show-o: One single transformer to unify multimodal understanding and generation. *arXiv preprint arXiv:2408.12528*, 2024. arXiv:2408.12528.
- [20] Jiacheng Ye, Shansan Gao, Shansan Gong, Lin Zheng, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Diffusion of thoughts: Chain-of-thought reasoning in diffusion language models. *arXiv preprint arXiv:2402.07754*, 2024. arXiv:2402.07754.
- [21] Wenhao Zhang et al. Time is a feature: Exploiting temporal dynamics in diffusion language models. *arXiv preprint arXiv:2508.09138*, 2025.
- [22] Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *arXiv preprint arXiv:2504.12216*, 2025. arXiv:2504.12216.
- [23] Chunting Zhou, Lili Yu, Arun Babu, Kushal Tirumala, Michihiro Yasunaga, Leonid Shamis, Jacob Kahn, Xuezhe Ma, Luke Zettlemoyer, and Omer Levy. Transfusion: Predict the next token and diffuse images with one multi-modal model. *arXiv preprint arXiv:2408.11039*, 2024. arXiv:2408.11039.

A Pre-Registration Scorecard

Four predictions were registered in `e2/PROTOCOL.md` before training began. Two hold cleanly, one is violated in the unexpected positive direction, and one is mixed (capacity-dependent in opposite directions across planners). Table 15 reproduces the scorecard.

Table 15: Pre-registration scorecard. v2 columns evaluated at v2 capacity (4/7 FFN modules, 10M LoRA params); v3 columns at v3 capacity (7/7 modules, 22M params). Square brackets are exact (Clopper–Pearson) binomial 95% CIs at $N=200$. For predictions #1 and #2, the 80% pre-registered target sits inside the v3 CI on test.

Prediction	v2	v3	Final
#1 $c2c \geq 80\%$	70.5% [63.7, 76.7]	79.0% [72.7, 84.4]	HOLDS
#2 base $c_{maj} b=5$ within ± 1 pp of 80%	81.5% [75.4, 86.6] (dev)	79.0% [72.7, 84.4] (test)	HOLDS
#3 $c_{majc} \leq c_{maj} + 1$ pp (no double-dip)	n/a	+3 pp (good direction)	VIOLATED UPWARD
#4 planner-quality threshold shifts down	hold, $\Delta = +7$ pp	inverts, $\Delta = -11$ pp	MIXED
<i>Post-hoc, registered 2026-05-03 in <code>phase2/proposals/verifier-based-aggregation.md</code>:</i>			
#5 per-branch verifier $\geq 83\%$ on $N=200$ held-out	n/a	best peer-class 76.5%	VIOLATED DOWNWARD
#5b frontier judge + CoT $\geq 83\%$ on $N=500$ extension	n/a	85.3% [81.9, 88.2] (oracle 86.3%)	HOLDS

Reading. Two clean holds (#1 with the registered 80% target inside the v3 [72.7, 84.4] CI; #2 with 80% inside the test [72.7, 84.4] CI). One violation in the unexpected positive direction (#3,

compositionality of `cmajc` on top of `c2c`). One mixed result (#4, capacity-dependent in opposite directions across planners): prediction holds at v2 capacity, inverts at v3 capacity. The inversion is itself a finding (§4) and is what made us promote planner-content trust to its own axis.

What we did not pre-register (originally). The diversity-expansion finding (§5) was not pre-registered as a positive prediction. Pre-registration #3 in the original protocol was that *Track 2 would not damage cmaj diversity* (a no-regression prediction). The actual result — diversity *expands* after Track 1, with 5/5-same rate dropping from 51.5% to 47.5% on test — falls outside the predicted scenarios. We treat it as a positive surprise and flag it as a non-pre-registered finding.

Post-hoc verifier prediction. Predictions #5 and #5b were registered after the original 4-prediction protocol but before the verifier sweeps and judge calls were run, in `phase2/proposals/verifier-based-` on 2026-05-03. #5 (a peer-class verifier $\geq 83\%$ on $N=200$ held-out) is violated downward: the strongest peer-class architecture across 17 tested reaches 76.5% (§8.3). #5b (a frontier judge with chain-of-thought $\geq 83\%$ on the $N=500$ extension) holds at 85.3% (§8.6). The two predictions were registered together because they were the alternative designs we wanted to disambiguate empirically; the resulting VIOLATED-DOWN/HOLDS pair is the body of §8.

Discipline note. Track 1 v3 was pre-committed as the headline before its `cmaj` number landed. At the time the framing decision was written (plan file, ~12:05 UTC), the v3 `cmaj` eval was mid-run at step 162/200. The final v3 `cmaj` test number (79.5% [73.2, 84.9]) came in below v2’s 81.5% [75.4, 86.6] on dev. Resisting the temptation to retreat to v2 as the headline post-hoc was the discipline cost of pre-registration; we report v3 as the headline regardless. We note that the two CIs overlap heavily; the -2 pp difference is well within sampling error at $N=200$, so the framing decision did not depend on a difference our eval can resolve.

B Compute Spend

The experimental program ran on a single RTX 4090 spot/on-demand substrate from Runpod (\$0.20/hr), with one OpenRouter judge sweep inside the voting-rule-gap follow-up. Total spend across all phases is approximately \$20.5; Table 16 reproduces the per-phase breakdown. The four phases correspond directly to the body of the paper: Phase 1 for the three-axis decomposition (§3–§5) and consensus distillation (§6); Phase 2 for the voting-rule gap and verifier sweep (§8); Phase 2/3 for the K2 schedule toggle (§9), the mode router (§10), and the S0–S4 spike chain (Appendix D); and Phase 4 for the parallel aggregation negatives surfaced in §11.2 and the cross-family substrate audit underlying §9.5.

Reproducibility. The full experimental record (training scripts, eval scripts, LoRA adapters, derived datasets, raw per-problem JSONL outputs, and W&B run links) is publicly available. All four LoRA adapters are on the Hugging Face Hub under the `eren23/` namespace: `sfumato-llada-prefix-robust-v2`, `sfumato-llada-prefix-robust-v3`, `sfumato-llada-commit-v2`, and `sfumato-llada-commit-v3`. The two derived datasets are `eren23/sfumato-prefix-robust-gsm8k` (59,784 rows) and `eren23/sfumato-commit-` (109 rows).

C Hyperparameters

This appendix reproduces the exact configuration of all four LoRA adapters reported in the body, derived from `scripts/train_track1_lora.py` and `scripts/train_track2_commit.py`.

C.1 Common Settings

All four adapters share the following base settings:

C.2 Track 1 (Prefix-Robustness)

Both v2 and v3 train on the `eren23/sfumato-prefix-robust-gsm8k` dataset (7,473 GSM8K-train problems crossed with 8 prefix tiers, 59,784 rows total, 95/5 train/validation split). The LLaDA training objective is the standard masked-diffusion CE loss: sample $t \sim U(0, 1)$, set $p_{\text{mask}} = (1 - \epsilon)t + \epsilon$, mask response tokens with probability p_{mask} , replace with `MASK_ID`, and apply CE only on masked positions re-weighted by $1/p_{\text{mask}}$. Prompt tokens are never masked.

p_{mask} note. Track 1 v1 (not reported in the body except for the v1 catastrophe note in Appendix E) used $p_{\text{mask}} \sim U(\epsilon, 1 - \epsilon)$, which over-weighted very-high-mask cases at training time. At inference, where the diffusion sampler typically uncovers ~ 16 – 32 fresh tokens per sub-block at moderate p_{mask} , the v1 model defaulted to a learned copy-back-context heuristic (bigram repetition loops). v2 narrowed the range to $U(0.1, 0.5)$ to match the regime LLADA actually inhabits at inference. v3 inherits this fix.

C.3 Track 2 (Commit Adapter)

Both v2 and v3 train on the `eren23/sfumato-commit-mixture-gsm8k` dataset (109 rows, 104 train / 5 validation), with three buckets: 46 rescue, 58 preserve-disagreement, 5 pure-agreement. The trainer re-uses the LLADA CE loss from Track 1 but masks a configurable span: `answer-only` [`answer_start`, `answer_end`) for v2, full response [`prompt_len`, `response_end`) for v3.

Note on Track 2 v1. Track 2 v1 (briefly mentioned in §6.1) used `target_modules=[gate_proj, up_proj, down_proj]` — the standard Llama list, which only matches `up_proj` on LLADA’s actual block. This produced a 4.2M-parameter adapter that attached to 1/3 of FFN. Discovering this bug while debugging Track 2 led to inspecting the Track 1 modules, and to the v3 retrain on the LLADA-correct list. v1 results are not reported in the body except as the matched 70.5% c2c plateau on the v1/v2 row of Table 6.

Inference configuration. All evaluations use $k=64$ denoising steps and `gen_length=128` tokens organized as 4 sub-blocks of 32. Stochastic conditions (`cmaj`, `cmajc`) sample $b=5$ branches at $t=0.7$ and majority-vote on extracted numeric answers. Deterministic conditions (`C2`, `c2c`, `C2hint`, `C2empty`, `C3p` X) use $t=0$ greedy decoding.

C.4 Verifier Configurations (§8)

The peer-class verifiers in §8.3 all share a common evaluation protocol and differ only in the encoder/feature side.

The chat-LM mean-pool + MLP family (which produced the encoder-scaling trend) uses the prompt template "Problem: {q}\n\nSolution:\n{branch_text}", mean-pools the last-layer hidden states under the attention mask, and trains a two-layer MLP head ($d_{\text{enc}} \rightarrow 256 \rightarrow 1$) for ~ 100 s per fold on a single A6000. Encoder weights are frozen.

The frontier-judge baseline (§8.6) calls Claude Sonnet 4.5 [4] via the OpenRouter API [16] once per branch. The CoT-then-verdict prompt instructs the judge to re-derive the problem setup from the natural-language question before judging the candidate solution; the YES/NO prompt asks for a verdict only. We sweep each of the 1,051 branches in the $N=500$ substrate. Per-call timeout is 90 s; concurrency 32; retries on transport errors only. The driver lives at `phase2/spikes/strong-judge/judge_via_openrouter.py`.

D Spike Chain (S0–S4): Inference-Path Engineering

Five small infrastructure spikes were run alongside the Phase-2 science work to harden and accelerate the inference path. None of them changes the headline accuracy claims, but two of them (S0 batched generation, S4 Fast-dLLM v1 drop-in) substantially reduce wall-clock cost for any reproduction effort, and S2–S3 are referenced from the body sections (§9’s `MERGE_ADAPTER=0` note, the `logit_shift_norm` diagnostic field in the released trajectory traces). We summarise the verdicts here for completeness.

Validation note. S1’s first implementation called the per-branch “last numeric token” extractor on partial chains-of-thought. This fired the quorum on mid-CoT numbers (e.g., the “13” inside “ $16 - 3 = 13$ ”) and pruned dissenters incorrectly, dropping accuracy from 0.75 to 0.45 on a 20-problem smoke test. The fix was a strict final-answer extractor that matches only the canonical `Answer: N` or `####N` forms; with that in place, the early-exit went from harmful to safe-but-inert. This is documented because it is the kind of bug a reproducer might hit if they implement an early-exit primitive on top of the released substrate.

Reproducibility. The substrate code (`e4/diff_llada.py`, `e4/runner.py`, `e4/grade.py`, `e4/fast_dllm_adapter.py`) and the pre-registration plus implementation logs (`phase2/spikes/`) are public; spike-chain GPU spend was $\sim \$5$ (S0–S4 inclusive), included in the Phase-2/3 line of Appendix B.

E Open Exposures

This appendix lists the experiments and analyses that we did not close before this draft. None of the items below change the headline numbers, but they are flagged for revision and for reviewers who want to know what is missing.

Multi-seed variance: closed. Earlier drafts flagged the absence of multi-seed error bars on the headline numbers. The follow-up phase ran the `cmajc` (Track-1-v3 + commit-v3, $b=5$, $\tau=0.7$) headline at $N=100$ for seeds 1 and 2 in addition to the seed-0 paper headline at $N=200$. Three-seed mean 82.2% with sample standard deviation 0.29 pp (§8.2, Table 9) is tighter than the per-seed Wilson interval at $N=100$. The 82.5% paper headline is robust to seed selection.

ABL_B sanity probe: closed. The +0.0 pp result for ABL_B (Table 7; Track 1 v3 LoRA + commit-v3 with `COMMIT_N_BLOCKS=1`, full-response training) was consistent with either (i) the

adapter firing as designed but landing downstream of the already-pinned reasoning (the mechanism story we tell) or (ii) a silent loader failure where the adapter does not modify inference at all. A direct probe [1] compared the Track-1-v3-only output against the Track-1-v3-plus-commit-v3 output at $\tau=0$ on 5 spot-check GSM8K-dev problems. All 5/5 problems show a systematic format shift (####N $\rightarrow$ Answer: N); the commit adapter is loaded and modifies inference, ruling out the silent no-op reading. The mechanism story stands.

Per-problem error-set diff for v1 vs. v2 not done. Track 2 v1 and v2 both produce 141/200 correct on test (§6.1). Whether the per-problem error sets are identical or merely aggregate-equal is itself informative: identical sets support a stronger “the late-block bottleneck binds the failure mode to specific problems” reading, while non-identical sets support a weaker “different adapters fail and pass on different problems but the late-block bottleneck caps both at the same total” reading. The body makes the weaker claim.

LCH-centroid mechanism figure for Track 1 was not produced. A latent-cluster-heatmap-style figure separating prefix-robust LoRA representations from base LLADA representations would harden the structural-separation thesis from behavioural to mechanistic. The JVP feasibility spike failed because PEFT [14] wraps LoRA in a `lora_magnitude_vector` ModuleDict that requires a custom JVP shim. Deferred to a follow-up.

Track 1 v1 catastrophe is documented in e2/RESULTS.TRACK1.md but not in the body. v1 used $r=16$, $\alpha=32$, learning rate $1e-4$, $p_{\text{mask}} \sim U(\epsilon, 1-\epsilon)$, and 14k steps; the resulting adapter regressed C2 from 74% to 59.5% (-14.5 pp) via bigram-repetition mode collapse (the model learned a copy-back-context heuristic at training time and defaulted to it at inference). v2 fixed this with four hyperparameter changes (smaller r , gentler LR, narrower p_{mask} range, fewer steps). We mention v1 only in passing in Appendix C; the contrast with v2 is a publishable masked-diffusion SFT pitfall on its own and is preserved in the source draft for a follow-up.

No cross-task transfer. All experiments are on GSM8K-test [6]. The diversity-expansion finding in §5 in particular is suspicious-looking on a single benchmark and deserves a multi-task replication (MATH-Easy, ARC-Challenge are the natural next targets).

No comparison to BD3-LMs and DiffuLLaMA on the same eval. The literature claims we discuss in §11 are reconciled *by reading* other authors’ results through the three-axis frame, not by re-running their adapters under our prefix conditions. A formal multi-DDLM ablation (BD3-LMs [5], DiffuLLaMA [8], LLADA, possibly hybrid multimodal models [13, 19, 23]) on the same prefix-tier dataset would be the cleanest falsification target for the three-axis framing. Out of scope for this draft.

Diversity-mechanism test deferred. Whether the diversity uptick in §5 is an implicit-temperature effect or a real content-diversity effect is open. The cheap test (base LLADA at $t \in \{0.8, 0.9, 1.0\}$ on the same eval, read the 5/5-same rate) was deferred to revision.

Temporal self-consistency: closed (negatively). A schedule-conditional temporal-SC variant on top of `cmajc` is reported in §11.2. Zhang et al. [21]’s exponential-decay weighting on diffusion LMs is the direct prior; we did not match or extend it on LLADA-class samplers, and a positive temporal-SC result under their weighting on this substrate remains open.

Reranker baseline: closed (negatively). Earlier drafts flagged a small classifier over 5 `cma`j branches as a compute-time alternative to v3’s param-time commit. §8 reports the result: 17 peer-class verifier architectures (text-bag, 0.5B–72B chat-LMs, embedding and reward models, process-feature MLPs, step-PRMs, symbolic arithmetic) all underperform majority vote at our substrate scale, with three robust negative findings (§8.4) and a monotone but insufficient encoder-scaling trend (§8.5). The reranker direction is not closed in the absolute sense — a 32B+ encoder, a process-reward verifier on Workstream-C traces, or PRM800K-style step-level supervision are still candidate Phase-3 paths — but the simplest version of the hypothesis (small per-branch classifier substitutes for param-time commit) is falsified.

Table 16: Compute spend. The original three-axis work reproduces in approximately one GPU-day on a single RTX 4090 (\$0.20/hr). The voting-rule-gap follow-up adds GPU substrate, peer-class verifier sweeps, and OpenRouter Claude Sonnet 4.5 calls (4 sweeps $\times$ $\sim$ 1,051 branch judgements each). The K2 sweep, mode-router bandit, and S0–S4 spike chain were batched onto two pods over a single night.

Phase	Cost (USD)
<i>Three-axis + distillation (Phase 1):</i>	
Track 1 v1 train + eval	\$0.80
Track 1 v2 train + full eval suite	\$0.50
Track 1 v3 train + eval suite	\$0.30
Track 2 v1 + v2 train + eval	\$0.55
Track 2 v3 train + eval	\$0.20
Disentangling ablations (ABL_A, ABL_B)	\$0.20
Diagnostic experiments + branch-agreement falsifier	\$0.50
QWEN2.5- self-consistency baseline	\$0.10
Buffer (debug, redos)	\$0.35
<i>Voting-rule gap (§8):</i>	
Substrate $N=200$ <code>cmaj</code> $b=5$ (v3 LoRA) + multi-seed <code>cmajc</code>	\$0.85
ABL_B sanity probe	\$0.05
Peer-class verifier sweeps (17 architectures, 0.5B–72B)	\$2.00
$N=500$ substrate extension and rich-feature dump	\$1.10
GPU-pod orchestration overhead and retries	\$3.50
OpenRouter Claude Sonnet 4.5 (4 sweeps $\times$ $\sim$ 1,051 calls)	\$2.00
<i>K2 sweep, mode router, spike chain (Phase 2/3):</i>	
K2 <code>COMMIT_N_BLOCKS</code> sweep ($k=0$ retry + $k=4$, multi-seed $k=3$ already on substrate)	\$1.30
S0–S4 spike chain (batched generation, ESC, merge toggle, logit-shift, Fast-dLLM)	\$2.50
D1 mode-router bandit (offline-replay, local sklearn LOOCV)	\$0.00
All-conditions sweep $\times$ showcase build (substrate for D1)	\$0.20
<i>Phase 4 (§11.2, §9.5):</i>	
Temporal-SC harvest + schedule-conditional vote scan	\$0.95
Schedule-RLHF mini-pilot (50-step GRPO, MATH-500 $N=20$ paired)	\$1.00
Schedule-RLHF mid-pilot (200-step GRPO + KL anchor, MATH-500 $N=50$ paired)	\$1.30
LLaDA-1.5 base swap (frozen v3 LoRAs)	\$0.30
DiffuLLaMA cross-family substrate audit (CPU only)	\$0.00
Total	$\sim$ \$20.5

Table 17: Common settings across all four LoRA adapters.

Setting	Value
Base model	GSAI-ML/LLaDA-8B-Instruct [15]
Precision	bfloat16
Tokenizer	<code>AutoTokenizer.from_pretrained(MODEL_NAME, trust_remote_code=True)</code>
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.01)
LR schedule	cosine decay with linear warmup
Gradient clip (max norm)	1.0
Activation checkpointing	on
Mask token id	126336 (LLaDA <mask>)
LoRA framework	PEFT [14]
LoRA dropout	0.05
LoRA bias	none
Random seed	42
Hardware	1×RTX 4090 (spot, \$0.20/hr)
Pinned dependency	<code>transformers==4.46.3</code>

Table 18: Track 1 v2 and v3 training configurations. Only the `target_modules` list differs. v2’s list misses three of LLaDA’s actual block names (`o_proj` → `attn_out`, `gate_proj` → `ff_proj`, `down_proj` → `ff_out`), attaching to 4/7 intended modules. v3’s list is the LLaDA-correct 7/7.

Setting	Track 1 v2	Track 1 v3
LoRA r	8	8
LoRA α	16	16
Learning rate	$5e-5$	$5e-5$
Max steps	5,000	5,000
Warmup steps	200	200
Batch size $\times$ grad accum	1×4	1×4
p_{mask} range	$U(0.1, 0.5)$	$U(0.1, 0.5)$
Max sequence length	512	512
Trainable parameters	$\approx 10\text{M}$	$\approx 22\text{M}$
<code>target_modules</code>	[q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]	[q_proj, k_proj, v_proj, attn_out, ff_proj, up_proj, ff_out]
Modules attached to LLaDA	4/7	7/7
Final val/loss	0.017	comparable

Table 19: Track 2 v2 and v3 training configurations. Only `FULL_RESPONSE_LOSS` (training) and `COMMIT_N_BLOCKS` (inference) differ. Both v2 and v3 use the LLADA-correct FFN-only target list, applied to the last 8 layers of LLADA-8B’s 32 (24–31).

Setting	Track 2 v2	Track 2 v3
LoRA r	8	8
LoRA α	16	16
Learning rate	5e−5	5e−5
Epochs	10	10
Warmup steps	20	20
Batch size $\times$ grad accum	1×4	1×4
p_{mask} range	$U(0.3, 0.9)$	$U(0.3, 0.9)$
Max sequence length	512	512
target_modules	[ff_proj, up_proj, ff_out]	[ff_proj, up_proj, ff_out]
layers_to_transform	24–31 (8/32)	24–31 (8/32)
<code>FULL_RESPONSE_LOSS</code>	0 (answer span only)	1 (full response)
<code>COMMIT_N_BLOCKS</code> (inference)	1 (last block only)	3 (blocks 2–4)
Trainable parameters	13.6M	13.6M

Table 20: Common protocol for peer-class verifier sweeps.

Setting	Value
Substrate	GSM8K-test cmaj branches, $b=5$, $t=0.7$
Substrate size	1,750 branches (350 problems $\times$ 5)
Cross-validation	5-fold by problem_id (no leakage)
Label	branch-correct: extracted answer matches gold
Loss	binary cross-entropy
Metric	verifier top-1 accuracy vs cmaj majority vote
Pre-reg threshold	$\geq 83\%$ mean over folds

Table 21: Phase-2 spike chain: each spike was pre-registered before implementation in `phase2/spikes/<name>/PRE_REG.md`, then implemented and measured against the pre-registration criteria. Wall-clock numbers are at **cmaj** $b=5$ unless noted. *Partial* on S4 reflects a known incompatibility between the Fast-dLLM fast path and the commit-LoRA toggle path; the latter is silently bypassed when `FAST_DLLM=1`, so the speedup applies to **cmaj** but not yet to **cmajc**.

Spike	Mechanism	Result
S0	Branch batching: cmaj/cmajc/cmerge run as a single $(B, L+\text{gen})$ batch	$1.53\times$ wall-clock at $N=$
S1	ESC quorum-3 early-exit when ≥ 3 of 5 branches agree	accuracy unchanged; in
S2	On-demand merge_adapter/unmerge_adapter at the commit-LoRA toggle boundary	$\leq 1\%$ wall-clock cost; ru
S3	Per-position logit_shift_norm shadow forward, gated by <code>LOGIT_SHIFT_NORM=1</code> env flag	populates trajectory sch
S4	Fast-dLLM v1 drop-in shim (<code>e4/fast_dllm_adapter.py</code> , gated by <code>FAST_DLLM=1</code>)	$6.5\times$ wall-clock at C2 N